ZisK's Recursion[1]

Héctor Masip-Ardevol
hector@zisk.technology
ZisK Core Team

September 14, 2026

Abstract

This document analyzes the security of the recursive aggregation layer of ZisK. It first gives a self-contained statement of the relations proved by the AIR arguments, the leaf verifiers, the aggregation nodes and the Final Verifier. The bus argument that ties the AIR instances together is abstracted as an additive fingerprint of a weighted multiset of messages, and the per-node bookkeeping as accumulators over a commutative monoid; the LogUp running sum and the LtHash challenge accumulator of ZisK are instances of these notions.

It then proves that any adversary producing an accepting Final proof for a public statement can be turned into an extractor that outputs a typed collection of AIR witnesses that are locally satisfying and bus balanced. The bound is an explicit sum of the knowledge-soundness errors of the four argument systems, weighted by the size of the unrolled aggregation tree, the soundness error of the fingerprint, the preimage awareness of the challenge accumulator and the extractability of the trace commitment, in the random-oracle model. Two idealizations are made explicit, straight-line composition and preimage awareness of the accumulator, and the assumptions inherited from the backend together with the obligations left for a complete proof are recorded in a ledger.

[1]This document describes ZisK v0.16.1 [48cf7cc – 2026-20-03].
Source code available at `https://github.com/0xPolygonHermez/zisk`.

Contents

1. Preliminaries and Recursive Relations **2**

1.1. Notation and Parameters 2

1.2. Arithmetization 2

1.3. The Aggregation Tree 4

1.4. Interfaces 5

1.5. Accumulators 6

1.6. Recursive Relations 7

1.7. ZisK Instantiation 9

2. Security **10**

2.1. Overview 10

2.2. Definitions 11

2.3. From the Root Check to Bus Balancing 13

2.4. Lemmas: Descending the Aggregation Tree 16

2.5. Main Theorem 19

2.6. Discussion 21

References **23**

1. Preliminaries and Recursive Relations

This section fixes the notation used in the security analysis and restates, in the exact form the proofs consume: the arithmetization layer (AIRs, interactions and buses), the aggregation tree and the interfaces of its cryptographic components (trace commitments, the multiset hash and key derivation), the three accumulators the tree carries, and the four relations attested by its nodes. ZisK's concrete choices are collected in Section 1.7.

1.1. Notation and Parameters.

Let $\mathbb{F}$ be a prime field of order p and $\mathbb{K}$ an extension field of $\mathbb{F}$. $\mathbb{Z}_p^n$ denotes the additive group of n-vectors of integers modulo p, the same set as $\mathbb{F}^n$ viewed as a $\mathbb{Z}$-module. For integers $a \le b$, $[a, b]$ and $[a, b)$ are the closed and half-open integer intervals and $[b] := [1, b]$. Field elements are identified with integers in $[0, p)$ when needed. λ is the security parameter; "PPT" means probabilistic polynomial time in λ, and $\mathsf{negl}(\lambda)$ and $\mathsf{poly}(\lambda)$ denote negligible and polynomially bounded functions. All algorithms have oracle access to the random oracles introduced in Section 1.4; Q bounds the total number of oracle queries an adversary makes.

The recursion is governed by one integer parameter, the *leaf-count exponent* $B \ge 1$: an aggregation tree may contain at most $2^B - 1$ leaves. We assume throughout that

$$2^{B+1} \le p, \quad 2^B < p, \quad 2^B \le \mathsf{poly}(\lambda) :$$

the first condition guarantees that sums of two range-checked counters never wrap around in $\mathbb{F}$, the second is the cardinality restriction required by the multiset hash, and the third makes the number of leaves, and hence the running time of the extractor, polynomial.

1.2. Arithmetization.

Definition 1.1 (AIR and Witness). An *algebraic intermediate representation* (AIR) of width W, depth D and V values is a finite set of *constraint polynomials*

$$\mathcal{C} \subset \mathbb{F}\Big[X_1^{(0)}, ..., X_W^{(0)}, ..., X_1^{(D)}, ..., X_W^{(D)}, Y_1, ..., Y_V\Big].$$

A *witness* of height N for $\mathcal{C}$ is a pair $w = (\boldsymbol{T}, \boldsymbol{a})$ with $\boldsymbol{T} \in \mathbb{F}^{N \times W}$, the *trace*, and $\boldsymbol{a} \in \mathbb{F}^V$, the *AIR values*. At row $r \in [N]$ the variable $X_j^{(k)}$ takes the value $T_j(r + k)$, with row indices taken modulo N, and Y_j takes the value a_j. The witness *satisfies* $\mathcal{C}$ if every constraint polynomial evaluates to zero at every row.

Definition 1.2 (Interaction). An *interaction* of an AIR $\mathcal{C}$ on the bus $b \in \mathbb{F}$ is a tuple $\iota = (b, m_\iota, \boldsymbol{e}_\iota)$, where the *multiplicity* m_ι and the *message* components $e_{\iota,1}, ..., e_{\iota,\ell_b}$ are polynomials over $\mathbb{F}$ in the variables of $\mathcal{C}$. Given a witness w and a row r, we write $m_\iota(w, r) \in \mathbb{F}$ and $\boldsymbol{e}_\iota(w, r) \in \mathbb{F}^{\ell_b}$ for their evaluations at row r. The set of bus identifiers in use is $\mathcal{B}$; every interaction on bus b has the common message length ℓ_b, and $\ell := \max_{b \in \mathcal{B}} \ell_b$.

Throughout, we fix a positive integer M and an indexed family of *AIR types*

$$\mathfrak{A} = \{(\mathcal{C}_\tau, \mathcal{I}_\tau)\}_{\tau \in [M]},$$

where each $\mathcal{C}_\tau$ is an AIR of width W_τ, depth D_τ and V_τ values and $\mathcal{I}_\tau$ is a finite set of interactions of $\mathcal{C}_\tau$. Given a fixed trace height N_τ, we write $\mathcal{W}_\tau := \mathbb{F}^{N_\tau \times W_\tau} \times \mathbb{F}^{V_\tau}$ for the witness space of type τ and fix the public-statement space $\mathcal{P} := \mathbb{F}^k$.

Definition 1.3 (Column Classes and Public Bindings). The description of an AIR $\mathcal{C}$ of width W and V values, used at height N, partitions its column indices as $[W] = J^{\mathrm{fix}} \sqcup J^{\mathrm{cach}} \sqcup J^{\mathrm{wit}}$ and carries:

1. The *fixed columns* J^{fix}, whose values are prescribed as part of the description of $\mathcal{C}$ and hence known to both the prover and the verifier.

2. The *cached columns* J^{cach}, whose values are computed once and reused across different proofs. Their values are not part of the description; instead, a commitment to them (Definition 1.9) travels in the public statement at the coordinates designated by a *cached-commitment binding* $\gamma : [m] \to [k]$.
3. The *witness columns* J^{wit}, computed afresh for every proof and only known in full to the prover.
4. A *public binding map* $\beta : \mathrm{dom}(\beta) \to [k]$ from a set $\mathrm{dom}(\beta) \subset [V]$ to coordinates of the public statement.

The *prescribed data* of $\mathcal{C}$ is the family $\boldsymbol{d} = \big(d_j\big)_{j \in J^{\mathrm{fix}}}$ of fixed columns $d_j \in \mathbb{F}^N$. For a trace $\boldsymbol{T}$ and a set of columns J we write $\boldsymbol{T}|_J$ for the submatrix of the columns in J.

Each AIR type carries this data; we index it by the type, as in J^{fix}_τ, $\boldsymbol{d}_\tau$ and β_τ.

Definition 1.4 (Local Satisfaction). For $\tau \in [M]$, $\boldsymbol{p} \in \mathcal{P}$ and $w = (\boldsymbol{T}, \boldsymbol{a}) \in \mathcal{W}_\tau$, the predicate $\mathrm{Local}_\tau(\boldsymbol{p}, w)$ holds if and only if

1. w satisfies $\mathcal{C}_\tau$.
2. $\boldsymbol{T}$ agrees with the prescribed data on the fixed columns: $T_j(r) = d_{\tau,j}(r)$ for every $j \in J^{\mathrm{fix}}_\tau$ and $r \in [N_\tau]$.
3. $\mathrm{Bind}_\tau(\boldsymbol{p}, w)$: $a_j = p_{\beta_\tau(j)}$ for every $j \in \mathrm{dom}(\beta_\tau)$.
4. The cached columns open the commitment carried in $\boldsymbol{p}$:

$$\mathsf{Commit}_{\mathrm{cach}}\big(\boldsymbol{T}|_{J^{\mathrm{cach}}_\tau}\big) = \big(p_{\gamma_\tau(1)}, ..., p_{\gamma_\tau(m)}\big).$$

Fixed columns are part of the description of $\mathcal{C}_\tau$, hence of the verification key of type τ. Cached columns are not: they are computed once per program, committed with $\mathsf{Commit}_{\mathrm{cach}}$, and the commitment travels in the public statement, where the description of $\mathcal{C}_\tau$ designates its coordinates. The program being executed is therefore bound through $\boldsymbol{p}$ rather than hard-wired into the circuits, as the program code is in the step circuit of the reference architecture [Kad+26]: the program ROM and the boot state live in cached columns of the relevant AIR types, and whoever accepts a Final proof for $\boldsymbol{p}$ must check that the cached commitments carried in $\boldsymbol{p}$ are those of the intended program. Since every node of a valid tree carries the same $\boldsymbol{p}$ (Section 1.6), binding of $\mathsf{Commit}_{\mathrm{cach}}$ makes the cached columns of all leaves of one type equal.

Definition 1.5 (Message multiset of a witness). Let $\mathcal{V} := \big\{(b, v) : b \in \mathcal{B}, v \in \mathbb{F}^{\ell_b}\big\}$ be the set of *bus messages*. An $\mathbb{F}$*-weighted multiset* of messages is a finitely supported map $D : \mathcal{V} \to \mathbb{F}$; we write $|D|$ for the size of its support and add such maps pointwise. For $\tau \in [M]$ and $w \in \mathcal{W}_\tau$, the *message multiset* of w is

$$\mathrm{Msg}_\tau(w)(b, v) := \sum_{\iota \in \mathcal{I}_\tau, b_\iota = b} \sum_{r=1}^{N_\tau} m_\iota(w, r) \cdot \mathbf{1}[\boldsymbol{e}_\iota(w, r) = v].$$

Its support has at most $N_\tau \cdot |\mathcal{I}_\tau|$ elements.

A row that sends a message with positive multiplicity and a row of the same witness that receives it with negative multiplicity cancel in $\mathrm{Msg}_\tau(w)$: the multiset records the net flow of every message out of the witness.

Definition 1.6 (Bus fingerprint). A *bus fingerprint* $\mathcal{F}$ consists of a finite *key space* $\mathcal{X}$, an abelian group $\mathcal{G}_\mathcal{F}$ of *values*, a class $\mathcal{D}$ of *admissible* nonzero $\mathbb{F}$-weighted multisets of messages, and, for every key $\chi \in \mathcal{X}$, a map $\mathcal{F}_\chi$ from $\mathbb{F}$-weighted multisets of messages to $\mathcal{G}_\mathcal{F} \cup \{\bot\}$, such that

1. (*additivity*) $\mathcal{F}_\chi(0) = 0$, and $\mathcal{F}_\chi(D + D') = \mathcal{F}_\chi(D) + \mathcal{F}_\chi(D')$ whenever the right-hand side is defined;

2. (*soundness*) for every Λ and every adversary $\mathcal{A}$ that outputs a multiset before the key is drawn uniformly from $\mathcal{X}$, the *fingerprint advantage*

$$\mathsf{Adv}^{\mathsf{fp}}_{(\mathcal{F},\Lambda)}(\mathcal{A}) := \Pr\big[D \leftarrow \mathcal{A}; \chi \leftarrow \mathcal{X} : D \in \mathcal{D} \wedge |D| \le \Lambda \wedge \mathcal{F}_\chi(D) = 0\big]$$

is small. The fingerprint is *statistically sound* with error $\varepsilon_{\mathcal{F}}$ if $\mathsf{Adv}^{\mathsf{fp}}_{(\mathcal{F},\Lambda)}(\mathcal{A}) \le \varepsilon_{\mathcal{F}}(\Lambda)$ for every adversary, with $\varepsilon_{\mathcal{F}}$ non-decreasing, and *computationally sound* if the advantage is negligible for every PPT adversary.

The *bus contribution* of $w \in \mathcal{W}_\tau$ under the key χ is

$$\mathrm{BusCont}_\chi(\mathcal{I}_\tau, w) := \mathcal{F}_\chi(\mathrm{Msg}_\tau(w)) \in \mathcal{G}_{\mathcal{F}} \cup \{\perp\}.$$

A fingerprint is an additive summary of a multiset that vanishes on an admissible nonzero multiset only for few keys, or only for keys no efficient adversary can anticipate. The relations of Section 1.6 are stated for an arbitrary fingerprint; ZisK's, LogUp, is described in Section 1.7.

Definition 1.7 (Typed witness collection). A *typed witness collection* is a finite list

$$L = \big((\tau_1, w_1), ..., \big(\tau_{|L|}, w_{|L|}\big)\big), \quad \tau_i \in [M], \quad w_i \in \mathcal{W}_{\tau_i}.$$

Types and witnesses may repeat, and each entry counts separately; the order of the list carries no meaning. The *number of bus terms* of L is $\Lambda(L) := \sum_{i=1}^{|L|} N_{\tau_i} \cdot \big|\mathcal{I}_{\tau_i}\big|$, and we set $\Lambda_{\max} := (2^B - 1) \cdot \max_{\tau \in [M]} N_\tau |\mathcal{I}_\tau|$, so that $\Lambda(L) \le \Lambda_{\max}$ whenever $|L| < 2^B$.

Definition 1.8 (Balance of a witness collection). The *signed multiplicity* of a typed witness collection L is the $\mathbb{F}$-weighted multiset

$$\mathcal{M}_L := \sum_{i=1}^{|L|} \mathrm{Msg}_{\tau_i}(w_i),$$

that is,

$$\mathcal{M}_L(b, v) = \sum_{i=1}^{|L|} \sum_{\iota \in \mathcal{I}_{\tau_i}, b_\iota = b} \sum_{r=1}^{N_{\tau_i}} m_\iota(w_i, r) \cdot \mathbf{1}[\boldsymbol{e}_\iota(w_i, r) = v].$$

The collection is *balanced*, written $\mathrm{Balanced}(L)$, if $\mathcal{M}_L = 0$, i.e. $\mathcal{M}_L(b, v) = 0$ for every bus $b \in \mathcal{B}$ and every message $v \in \mathbb{F}^{\ell_b}$. Note that $|\mathcal{M}_L| \le \Lambda(L)$.

Note that multiplicities are field elements: $\mathrm{Balanced}(L)$ does not by itself assert that every message is sent and received the same number of times as integers, since p copies of a message cancel. We return to this point in Remark 2.4.

1.3. The Aggregation Tree.

A ZisK proof is a tree. Its leaves are the AIR instances of an execution: for each type $\tau \in [M]$ the prover produces some number of witnesses in $\mathcal{W}_\tau$, one per segment of that type, and proves each with an argument system for the AIR of the type. Directly above every AIR proof sits a *leaf verifier*, a circuit that verifies the AIR proof and re-exposes its statement in a common format. Above the leaf verifiers, *aggregation nodes* verify two child proofs each, so the tree is binary; it need not be balanced, because a node is proved as soon as its two children are available. At the top, a single *Final Verifier* verifies the root of the aggregation tree and performs the global checks; its statement, the public statement $\boldsymbol{p}$, is the only thing an external verifier sees. Figure 1 shows an example with three leaves.

Three quantities travel up the tree and are combined at every aggregation node: the number of leaves, an accumulation of the leaves' commitments, and an accumulation of the leaves' bus contributions. They

are the *accumulators* of Section 1.5; the counter is range-checked at every node, and the Final Verifier checks one condition on each.

The order of operations matters for the bus argument. Bus contributions are fingerprints under a key χ, and the key must be fixed before any leaf is proved yet must not be under the prover's control. ZisK derives it from the leaves themselves: the prover commits to every witness, accumulates the typed commitments with the multiset hash $\mathcal{H}$, derives χ from the accumulated value with DeriveKey, and only then proves the leaves with χ as a public input. The tree recomputes the accumulation as it aggregates, and the Final Verifier checks that the key the leaves used is the one derived from what was actually aggregated. Because the shape of the tree is not known when the key is derived, the accumulation must not depend on it; this is why the accumulators below are commutative monoids.

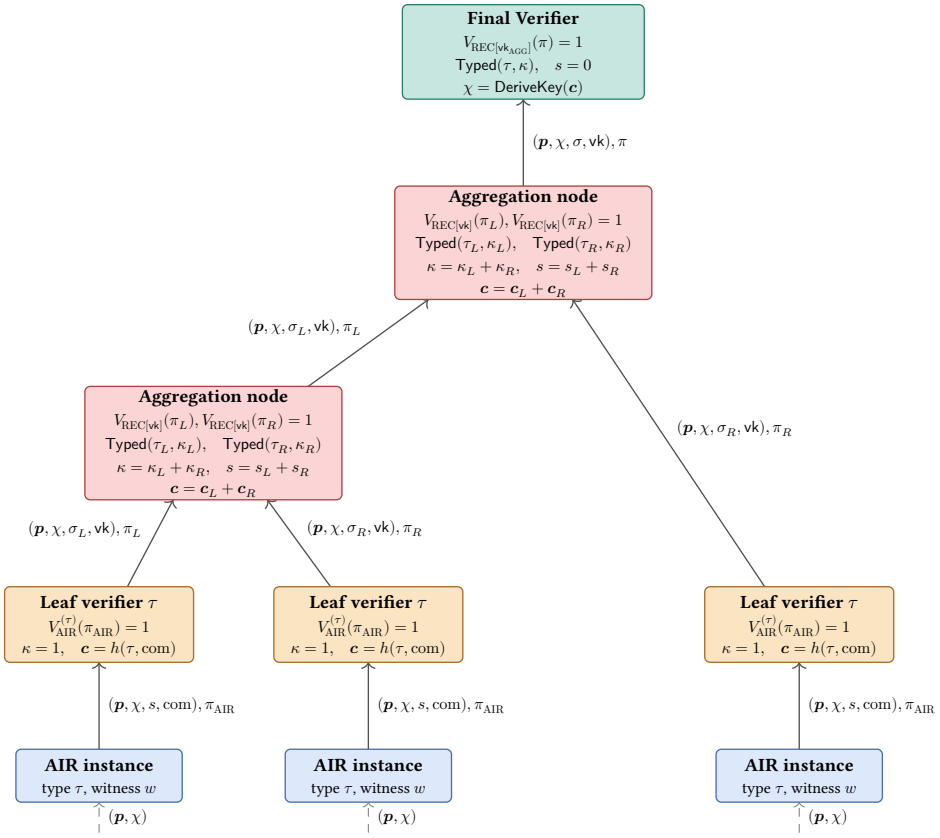


FIGURE 1. An aggregation tree with three AIR instances. Each arrow carries the statement and the proof a node produces, which the node above takes as witness: an AIR instance of type τ proves $(\boldsymbol{p}, \chi, s, \mathrm{com})$; a leaf verifier re-exposes it as $(\boldsymbol{p}, \chi, \sigma, \mathsf{vk})$ with $\sigma = (\tau, 1, \boldsymbol{c}, s)$; an aggregation node adds its children's accumulators, dispatches on their types and declares type $M + 1$; the Final Verifier fixes $\mathsf{vk} = \mathsf{vk}_{\mathrm{AGG}}$, checks the root predicates and exposes only $\boldsymbol{p}$. The dashed arrows are the public inputs common to every leaf.

1.4. **Interfaces.**

A *non-interactive argument system* Π for a relation $\mathcal{R}$ consists of a prover and a deterministic *verifier* V, together with a verification key vk_Π produced at setup, where $V(\mathsf{vk}_\Pi, x, \pi) \in \{0, 1\}$ and the value 1 means that the proof π is accepted for the statement x. We write $V_X(x, \pi) := V(\mathsf{vk}_X, x, \pi)$ for the verifier of the system attesting to the relation $\mathcal{R}_X$, and keep the explicit form $V(\mathsf{vk}, x, \pi)$ where the key is a variable. The relations below refer to argument systems only through their verifiers, so the prover is never named again. The relations of Section 1.6 refer to argument systems only through their verifiers, and to

the following fixed public maps only through their signatures. Their security properties are defined where they are used, in Section 2, and ZisK's choices are given in Section 1.7. Two of the maps are commitments, in the following sense.

Definition 1.9 (Deterministic Commitment Scheme). A *deterministic commitment scheme* Com consists of a set of *messages*, a finite set of *commitments*, and a polynomial-time computable function Commit from the former to the latter. It is *binding* if for every PPT adversary $\mathcal{A}$

$$\mathsf{Adv}^{\mathsf{bind}}_{\mathsf{Com}}(\mathcal{A}) := \Pr\big[(m, m') \leftarrow \mathcal{A}\big(1^\lambda\big) : m \neq m' \wedge \mathsf{Commit}(m) = \mathsf{Commit}(m')\big] \leq \mathsf{negl}(\lambda).$$

Since Commit is deterministic, binding is collision resistance of Commit. When the messages are vectors or matrices over $\mathbb{F}$, the scheme is a *vector commitment* if it also offers openings of individual positions.

The following interface elements are used throughout:

- **Carriers.** A finite key space $\mathcal{X}$ and an abelian group $\mathcal{G}_{\mathcal{F}}$ of fingerprint values (Definition 1.6), and a finite abelian *accumulator group* $\mathcal{G}_{\mathcal{H}}$.
- **Trace commitment.** A deterministic commitment scheme Com with message space $\bigcup_{\tau\in[M]} \mathcal{W}_\tau$ and commitment space $\{0,1\}^*$. A commitment $\mathrm{com} = \mathsf{Commit}(\boldsymbol{T}, \boldsymbol{a})$ covers every column of the trace and every AIR value. The argument system for an AIR type takes the key $\chi \in \mathcal{X}$ as a public input, so the witness is key-independent.
- **Cached-column commitment.** A deterministic commitment scheme $\mathsf{Com}_{\mathrm{cach}}$ with message space $\bigcup_{\tau\in[M]} \mathbb{F}^{N_\tau \times |J^{\mathrm{cach}}_\tau|}$ and commitment space $\mathbb{F}^m$. Its output for the cached columns of a type is not part of a verification key but of the public statement (Definition 1.3).
- **Hash-to-group and multiset hash.** $h : [M] \times \{0,1\}^* \to \mathcal{G}_{\mathcal{H}}$ maps a *typed commitment* (τ, com) to a group element, and

  $$\mathcal{H}(\mathcal{S}) := \sum_{x \in \operatorname{supp} \mathcal{S}} \mathrm{mult}_{\mathcal{S}}(x) \cdot h(x) \in \mathcal{G}_{\mathcal{H}}$$

  is the induced multiset hash on finite multisets $\mathcal{S}$ of typed commitments, where $\mathrm{mult}_{\mathcal{S}}(x)$ is the multiplicity of x in $\mathcal{S}$. $\mathcal{H}$ is additively homomorphic over multiset union.
- **Key derivation.** $\mathsf{DeriveKey} : \mathcal{G}_{\mathcal{H}} \to \mathcal{X}$, a fixed public function of the accumulated value.

1.5. Accumulators.

Definition 1.10 (Accumulator). An *accumulator* over the aggregation tree consists of a commutative monoid of *values*, a *leaf contribution* assigning a value to every leaf, a *child check*, a predicate on values that every aggregation node evaluates on the value of each of its children, and a *root check*, a predicate on values that the Final Verifier evaluates on the value of the root of the aggregation tree. The value of a leaf is its contribution, and the value of an aggregation node is the sum of the values of its two children. By associativity and commutativity, the value of any node is the sum of the contributions of the leaves below it, whatever the shape of the subtree.

The recursion carries three accumulators. Their values at a node, together with the node's type, form the *recursive statement*

$$\sigma = (\tau, \kappa, \boldsymbol{c}, s),$$

where the type $\tau \in [M+1]$ is the AIR type for a leaf verifier and the marker $M+1$ for an aggregation node. We write $\sigma.\tau$, $\sigma.\kappa$, $\sigma.\boldsymbol{c}$ and $\sigma.s$ for the components of a recursive statement.

Accumulator	**Monoid**	**Leaf contribution**	**Child check**	**Root check**
leaf counter κ	$(\mathbb{F}, +)$	1	$\kappa_X \in [1, 2^B)$	$\kappa \in [1, 2^B)$
challenge accumulator $\boldsymbol{c}$	$(\mathcal{G}_{\mathcal{H}}, +)$	$h(\tau, \mathrm{com})$	$-$	$\chi = \mathsf{DeriveKey}(\boldsymbol{c})$

Accumulator	Monoid	Leaf contribution	Child check	Root check
bus partial sum s	$(\mathcal{G}_{\mathcal{F}}, +)$	$\mathcal{F}_{\chi}(\mathrm{Msg}_{\tau}(w))$	$-$	$s = 0$

TABLE 1. The three accumulators of the recursion.

The counter is added in $\mathbb{F}$. Its child check is the typing predicate of Definition 1.13, which every aggregation node applies to both children and the Final Verifier applies to the root; it places every counter in $[1, 2^B)$, and $2^{B+1} \leq p$ then rules out wrap-around, so the field sums are integer sums and no node has 2^B or more leaves below it. The other two accumulators are checked only at the root. The value of the challenge accumulator at a node is $\mathcal{H}$ of the multiset of typed commitments below it, by the homomorphism of $\mathcal{H}$; the value of the bus partial sum is $\mathcal{F}_{\chi}$ of the summed message multisets below it, by the additivity of $\mathcal{F}$, so at the root it equals $\mathcal{F}_{\chi}(\mathcal{M}_L)$ for the collection L of all leaves.

An update rule that is not associative and commutative would make the root value depend on the shape of the tree, which is not known when the key is derived (Section 1.3); a hash-tree accumulation $\boldsymbol{c} = H(\boldsymbol{c}_L, \boldsymbol{c}_R)$, for instance, would force the prover to fix the tree before deriving χ. Writing every update as the monoid sum therefore loses no generality.

1.6. **Recursive Relations.**

We write a relation as $\mathcal{R} = \{(x; w) \mid \varphi(x, w)\}$ with public statement x, witness w and *defining predicate* φ; in the definitions below, φ is the conjunction of the clauses displayed to the right of the bar. Every node statement below the Final Verifier carries the public statement $\boldsymbol{p} \in \mathcal{P}$, a key $\chi \in \mathcal{X}$, a recursive statement σ and a verification key vk. We proceed from the leaves to the root, naming after each relation the argument system that proves it, since the relation above uses its verifier.

Definition 1.11 (AIR relation). For $\tau \in [M]$,

$$\mathcal{R}^{(\tau)}_{\mathrm{AIR}} = \left\{ (\boldsymbol{p}, \chi, s, \mathrm{com}); (\boldsymbol{T}, \boldsymbol{a}) \;\middle|\; \begin{array}{ll} & \mathrm{Local}_{\tau}(\boldsymbol{p}, (\boldsymbol{T}, \boldsymbol{a})) \\ \wedge & s = \mathrm{BusCont}_{\chi}(\mathcal{I}_{\tau}, (\boldsymbol{T}, \boldsymbol{a})) \\ \wedge & \mathsf{Commit}(\boldsymbol{T}, \boldsymbol{a}) = \mathrm{com} \end{array} \right\}$$

with $\boldsymbol{p} \in \mathcal{P}$, $\chi \in \mathcal{X}$, $s \in \mathcal{G}_{\mathcal{F}}$ and $(\boldsymbol{T}, \boldsymbol{a}) \in \mathcal{W}_{\tau}$.

The relation is in *commit-and-prove* form: the trace commitment is part of the statement. A proof for $\mathcal{C}_{\tau}$ already contains this commitment, so $V^{(\tau)}_{\mathrm{AIR}}$ simply reads it off the proof and compares it with the statement component; the point of exposing it is that the leaf verifier can then bind the challenge contribution to a value that is a function of the statement and witness rather than of an object that occurs in neither.

Let $\Pi^{(\tau)}_{\mathrm{AIR}}$ be an argument system for $\mathcal{R}^{(\tau)}_{\mathrm{AIR}}$, one per type, with verifier $V^{(\tau)}_{\mathrm{AIR}}$ and key $\mathsf{vk}^{(\tau)}_{\mathrm{AIR}}$.

Definition 1.12 (Leaf-verifier relation). For $\tau \in [M]$,

$$\mathcal{R}^{(\tau)}_{\mathrm{LEAF}} = \left\{ (\boldsymbol{p}, \chi, \sigma, \mathsf{vk}); (\mathrm{com}, \pi_{\mathrm{AIR}}) \;\middle|\; \begin{array}{ll} & \sigma = (\tau, 1, \boldsymbol{c}, s) \\ \wedge & V^{(\tau)}_{\mathrm{AIR}}((\boldsymbol{p}, \chi, s, \mathrm{com}), \pi_{\mathrm{AIR}}) = 1 \\ \wedge & \boldsymbol{c} = h(\tau, \mathrm{com}) \end{array} \right\}$$

The key vk carried in the statement is the propagated aggregation key, which is not used here.

In the terms of Section 1.5, a leaf verifier of type τ contributes 1, $h(\tau, \mathrm{com})$ and the bus contribution s certified by the AIR proof. Let $\Pi^{(\tau)}_{\mathrm{LEAF}}$ be an argument system for $\mathcal{R}^{(\tau)}_{\mathrm{LEAF}}$, with verifier $V^{(\tau)}_{\mathrm{LEAF}}$ and key $\mathsf{vk}^{(\tau)}_{\mathrm{LEAF}}$; its circuit hard-codes $\mathsf{vk}^{(\tau)}_{\mathrm{AIR}}$.

An aggregation node must verify children of every kind, leaf verifiers of any type and other aggregation nodes. It hard-codes the M leaf-verifier keys and selects among them by the child's type; for an aggregation child it uses a key that it does not hard-code but reads from its own statement.

Definition 1.13 (Recursive Dispatch). For a type $\tau \in [M+1]$ and a key vk, the *dispatched verifier* is

$$V_{\mathrm{REC}}^{(\tau)}[\mathsf{vk}](x,\pi) := \begin{cases} V_{\mathrm{LEAF}}^{(\tau)}(x,\pi) \text{ if } \tau \in [M], \\ V(\mathsf{vk},x,\pi) \text{ if } \tau = M+1, \end{cases}$$

and $V_{\mathrm{REC}}^{(\tau)}[\mathsf{vk}]$ rejects everything if $\tau \notin [M+1]$. The *typing predicate* on a type and a counter is

$$\mathsf{Typed}(\tau,\kappa) := (\tau \in [M] \wedge \kappa = 1) \vee (\tau = M+1 \wedge 2 \le \kappa < 2^B).$$

The dispatch is a function of the key vk carried in the statement, and the typing predicate is what makes the counter meaningful (Section 1.5).

Definition 1.14 (Aggregation relation). Write $x_X := (\boldsymbol{p}, \chi, \sigma_X, \mathsf{vk})$ for the statement of the child $X \in \{L,R\}$ and $\sigma_X = (\tau_X, \kappa_X, \boldsymbol{c}_X, s_X)$ for its recursive statement. Then

$$\mathcal{R}_{\mathrm{AGG}} = \left\{ (\boldsymbol{p},\chi,\sigma,\mathsf{vk}); ((\sigma_L,\pi_L),(\sigma_R,\pi_R)) \;\middle|\; \begin{array}{ll} & \sigma = (M+1,\kappa,\boldsymbol{c},s) \\ \wedge & \mathsf{Typed}(\tau_X,\kappa_X) \text{ for } X \in \{L,R\} \\ \wedge & V_{\mathrm{REC}}^{(\tau_X)}[\mathsf{vk}](x_X,\pi_X) = 1 \text{ for } X \in \{L,R\} \\ \wedge & \kappa = \kappa_L + \kappa_R \text{ and } \boldsymbol{c} = \boldsymbol{c}_L + \boldsymbol{c}_R \text{ and } s = s_L + s_R \end{array} \right\}$$

Two invariants of this relation are used repeatedly. First, the *propagation invariant*: both children are verified on statements that carry the parent's own $(\boldsymbol{p},\chi,\mathsf{vk})$, so every node of a valid tree carries the root's public statement, fingerprint key and verification key; in particular an aggregation child is verified under $V(\mathsf{vk},\cdot,\cdot)$ for the very vk the parent declares. Second, the *counter invariant*: Typed places both children's counters in $[1,2^B)$, and $2^{B+1} \le p$ makes $\kappa = \kappa_L + \kappa_R$ an identity of integers, so every child has $\kappa_X \le \kappa - 1$.

Let Π_{AGG} be an argument system for $\mathcal{R}_{\mathrm{AGG}}$, with verifier V_{AGG} and key $\mathsf{vk}_{\mathrm{AGG}}$. There is no circularity: $\mathcal{R}_{\mathrm{AGG}}$ is one ordinary relation over statements that contain an arbitrary string vk, the key $\mathsf{vk}_{\mathrm{AGG}}$ of the argument system for that relation is produced afterwards at setup, and the Final relation below fixes $\mathsf{vk} := \mathsf{vk}_{\mathrm{AGG}}$, at which point $V_{\mathrm{REC}}^{(M+1)}[\mathsf{vk}_{\mathrm{AGG}}] = V_{\mathrm{AGG}}$.

Definition 1.15 (Final relation). Fix the aggregation key $\mathsf{vk}_{\mathrm{AGG}}$ as a parameter. Then

$$\mathcal{R}_{\mathrm{FINAL}} = \left\{ \boldsymbol{p}; (\chi,\sigma,\pi) \;\middle|\; \begin{array}{ll} & \sigma = (\tau,\kappa,\boldsymbol{c},s) \text{ and } \mathsf{Typed}(\tau,\kappa) \\ \wedge & V_{\mathrm{REC}}^{(\tau)}[\mathsf{vk}_{\mathrm{AGG}}]((\boldsymbol{p},\chi,\sigma,\mathsf{vk}_{\mathrm{AGG}}),\pi) = 1 \\ \wedge & s = 0 \\ \wedge & \chi = \mathsf{DeriveKey}(\boldsymbol{c}) \end{array} \right\}$$

Only the public statement $\boldsymbol{p}$ survives at the root: the fingerprint key, the recursive statement and the child proof are witnesses. Dispatching on τ lets the Final Verifier accept a leaf-verifier proof directly when the execution fits in a single AIR instance.

Let Π_{FINAL} be an argument system for $\mathcal{R}_{\mathrm{FINAL}}$, with verifier V_{FINAL} and key $\mathsf{vk}_{\mathrm{FINAL}}$; this is the verifier run by the external party. The verification keys involved play different roles. Table 2 records them.

Key	**Selected by**	**Hard-coded in**
$\mathsf{vk}_{\mathrm{AIR}}^{(\tau)}$	the type τ of the leaf verifier circuit	$\Pi_{\mathrm{LEAF}}^{(\tau)}$
$\mathsf{vk}_{\mathrm{LEAF}}^{(\tau)}$	the child type $\tau_X \in [M]$ read from the child statement	Π_{AGG} (all M keys)
$\mathsf{vk}_{\mathrm{AGG}}$	the child type $\tau_X = M+1$; the key itself arrives as the public vk	Π_{FINAL} only

Key	Selected by	Hard-coded in
$\mathsf{vk}_{\mathrm{FINAL}}$	—	the external verifier

TABLE 2. Verification-key ledger. The circuit type τ selects which hard-coded key an aggregation node uses for a child; the propagated public vk is the aggregation verifier's own key, which no aggregation circuit can hard-code and which the Final Verifier fixes.

1.7. ZisK Instantiation.

The security analysis of Section 2 is stated for the following choices, which fix the carriers and the maps of Section 1.4. Four of them are built from one *underlying hash function*, and the analysis depends on that function only through the random-oracle model of Definition 2.4, never through its internals. ZisK currently instantiates it with Poseidon2 [GKS23], whose arithmetisation-friendly design makes the recursive circuits cheap; a general-purpose hash such as BLAKE3 [O'C+20] or SHA-2 [Nat15] is an equally valid instantiation, and switching among them changes none of the statements below. The only requirements are those the analysis names where it uses them: collision resistance of the digest, with at least 2λ bits (Section 2.6), and the random-oracle idealization.

Bus fingerprint. The key space is $\mathcal{X} = \mathbb{K}^2$, a key being a *challenge pair* $\chi = (\alpha, \gamma)$, and the values lie in $\mathcal{G}_{\mathcal{F}} = \mathbb{K}$. The fingerprint is LogUp [Hab22]: for $\chi = (\alpha, \gamma)$ the message (b, v) is *compressed* to $\hat{e}_\gamma(b, v) := b + \sum_{j=1}^{\ell_b} \gamma^j v_j \in \mathbb{K}$ and

$$\mathcal{F}_\chi^{\mathrm{LogUp}}(D) := \sum_{(b,v)\in \operatorname{supp} D} \frac{D(b,v)}{\alpha + \hat{e}_\gamma(b,v)},$$

with $\mathcal{F}_\chi^{\mathrm{LogUp}}(D) := \perp$ if some denominator vanishes. It admits every nonzero multiset, and Lemma 2.9 shows that it is statistically sound with $\varepsilon_{\mathcal{F}}(\Lambda) = \big(\ell \cdot \binom{\Lambda}{2} + \max(\Lambda - 1, 0)\big)/|\mathbb{K}|$. The AIR argument does not evaluate this closed form. It constrains a running sum over the rows of the trace, adding $m_\iota(w,r)/\big(\alpha + \hat{e}_\gamma(b_\iota, \boldsymbol{e}_\iota(w,r))\big)$ at every row through an explicitly constrained inverse, and exposes the final value s. Grouping the row terms by message shows that the exposed value equals $\mathcal{F}_\chi^{\mathrm{LogUp}}(\mathrm{Msg}_\tau(w))$ whenever every row denominator is nonzero. The circuit therefore proves a sub-relation of $\mathcal{R}_{\mathrm{AIR}}^{(\tau)}$: it additionally rejects when a row of zero multiplicity, or a pair of rows that cancel in the multiset, hits a pole. This only strengthens the statements of Section 2, since knowledge soundness for a sub-relation implies knowledge soundness for the relation.

Multiset hash. The accumulator group is $\mathcal{G}_{\mathcal{H}} = \mathbb{Z}_p^n$ with $n = 372$, and h and $\mathcal{H}$ are LtHash [Cla+03, Zis25]: h hashes the typed commitment with the underlying hash function and expands the digest into a pseudorandom vector of $\mathbb{Z}_p^n$, and $\mathcal{H}$ adds these vectors with multiplicity. Its collision resistance on multisets of cardinality below 2^B rests on the hardness of the short integer solution problem [Ajt96]; the analysis assumes the stronger preimage awareness (Definition 2.3), which is the second idealization of Remark 2.1. The cardinality bound is what the leaf counter enforces.

Key derivation. $\mathsf{DeriveKey}$ hashes the accumulated vector with the underlying hash function and reads the digest as $\alpha \in \mathbb{K}$; the second component γ is a further hash of α. The analysis treats the pair as a single random-oracle output (Definition 2.4) and suppresses the $(Q+1)/|\mathbb{K}|$ term for a derived α colliding with a previously queried input.

Trace commitment. Commit is a Merkle tree over the underlying hash function, so it is straight-line extractable in the random-oracle model (Definition 2.2).

Cached-column commitment. $\mathsf{Commit}_{\mathrm{cach}}$ is likewise a Merkle tree over the underlying hash function. The program ROM is the cached column group of the ROM AIR type, and its root is one of the program's public inputs, so a proof is bound to a program through $\boldsymbol{p}$. The analysis requires the leaf commitment $\mathrm{com} = \mathsf{Commit}(\boldsymbol{T}, \boldsymbol{a})$ to bind the cached columns as well, since the challenge accumulator

hashes com and not $\boldsymbol{p}$: if the leaf commitment covered the witness columns only, a prover could choose the ROM after seeing the key χ, and the fingerprint argument of Proposition 2.12 would not apply. Either com includes the cached commitment, or the latter must enter the challenge contribution by other means; this is a circuit requirement to be checked (Section 2.6).

2. Security

2.1. Overview.

We prove that the ZisK aggregation tree is *correct-witness extractable*: any adversary that produces an accepting Final proof for a public statement $\boldsymbol{p}$ can be used to extract, in a straight-line fashion, a typed witness collection L such that every witness satisfies its AIR and is consistent with $\boldsymbol{p}$, and every bus balances across the whole collection. The proof descends the aggregation tree of Section 1 layer by layer, reducing everything to the knowledge soundness of the four kinds of argument systems ($\Pi_{\mathrm{AIR}}^{(\tau)}$, $\Pi_{\mathrm{LEAF}}^{(\tau)}$, Π_{AGG} and Π_{FINAL}), the straight-line extractability of the trace commitment Com, the preimage awareness of the accumulator $\mathcal{H}$, and the soundness of the bus fingerprint.

The section follows the reference architecture of the EF zkVM security sprint [Kad+26] in structure as well as in method: an idealized-model remark, the definitions of the assumptions, a bridging proposition, one extraction game and lemma per layer of the recursion, and a main theorem whose bound is an explicit sum over those lemmas. Where ZisK departs from that architecture, the departure is flagged at the point where it matters.

Remark 2.1 (Idealized proof model). *The analysis is idealized in three respects.*

First, Definition 2.1 postulates a straight-line extractor that runs in essentially the time of the adversary. Because the argument systems are hash-based, the natural model is the random oracle model (ROM), in which such extractors exist for a single argument [BCS16]. Composing straight-line extraction across the layers of a recursion tree, however, requires every argument to remain knowledge-sound relative to the same oracle, and such relativized succinct arguments do not exist in general [BCG24]. In our analysis we idealize over this obstruction and assume that straight-line extraction composes.

Second, the bus argument is sound only if the multiset of typed commitments that the accumulator $\boldsymbol{c}$ stands for is fixed before the random oracle answers at $\boldsymbol{c}$. For the trace commitments this is the straight-line extractability of Merkle trees in the ROM (Definition 2.2). For the accumulator we assume the analogous property, preimage awareness (Definition 2.3): a straight-line extractor recovers, from the transcript at the time a value is announced, the multiset the adversary later opens it to. For the algebraic accumulator $\mathcal{H}$ this is an idealization. Distinct small multisets with equal hash exist in abundance, only the hardness of finding them protects the scheme, and recovering a multiset from its accumulator is a subset-sum problem, so no straight-line extractor is known; only collision resistance follows from standard assumptions. Remark 2.3 discusses the rigorous alternative.

Third, while the analysis produces explicit advantage bounds and reduction running times (Theorem 2.21), the two idealizations mean that these should be read as statements about the reduction structure (the recursion topology is sound and the relations compose) rather than as a concrete security level.

Remark 2.2 (Scope). *The target relation of this section speaks about AIR witnesses, not about guest executions: it is the recursion layer of the chain that leads from an accepting final proof to a correct execution of the guest program. Not covered here, and left as named obligations, are the semantic layer (that a balanced collection of locally satisfying witnesses encodes a single execution of the committed program, including the memory argument and continuation stitching), the knowledge soundness of the individual STARK arguments (a property of the backend, quantified by their own analysis), completeness, zero knowledge, and any wrapper or compression layers placed above the Final Verifier, each of which adds one more lemma of the kind proved for the leaf-verifier layer.*

2.2. **Definitions.**

Definition 2.1 (Knowledge soundness). A non-interactive argument system Π for a relation $\mathcal{R}$ is *knowledge-sound* if there exists a PPT algorithm $\mathcal{E}$, the *extractor*, such that for every PPT adversary $\mathcal{A}$:

$$\mathsf{Adv}^{\mathsf{ks}}_{\Pi}(\mathcal{A}) := \Pr\big[(x,\pi) \leftarrow \mathcal{A}\big(1^{\lambda}\big); w \leftarrow \mathcal{E}(x,\pi) : V(\mathsf{vk}_{\Pi}, x, \pi) = 1 \wedge (x;w) \notin \mathcal{R}\big] \leq \mathsf{negl}(\lambda),$$

the probability taken over the coins of $\mathcal{A}$ and $\mathcal{E}$ and the random oracles, to which all algorithms have access. We assume that the running time of $\mathcal{E}(x,\pi)$ is at most $\mathsf{Time}\big(\mathcal{A}\big(1^{\lambda}\big)\big) + \mathsf{poly}(\lambda)$ for every $\mathcal{A}$ and every (x,π) in the support of $\mathcal{A}$'s output: the extractor incurs an additive polynomial overhead over one evaluation of $\mathcal{A}$. We call $\mathsf{Adv}^{\mathsf{ks}}_{\Pi}(\mathcal{A})$ the *knowledge-soundness error* of $\mathcal{A}$ against Π.

Remark (Straight-line extraction). *The extractor of Definition 2.1 is straight-line: it receives only the statement and the proof and does not rewind the adversary. For succinct proofs this is not achievable in the plain model, and in the ROM it holds for one argument but does not compose across a recursion (Remark 2.1). The definition is a proxy that makes the reduction structure explicit; a concrete instantiation replaces it by ROM extractability with the overhead discussed there.*

Definition 2.2 (Straight-line extractable commitment). A deterministic commitment scheme Com (Definition 1.9) is *straight-line extractable* if there is a PPT algorithm $\mathcal{E}_{\mathsf{Com}}$ that, given a transcript $\mathcal{T}$ of random-oracle queries and answers and a value com, outputs a message or $\perp$, such that for every PPT adversary $\mathcal{A}$ that at some point during its execution announces a value com and at the end outputs a message m,

$$\mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{A}) := \Pr\big[\mathsf{Commit}(m) = \mathrm{com} \wedge \mathcal{E}_{\mathsf{Com}}(\mathcal{T}_{\leq t}, \mathrm{com}) \neq m\big] \leq \mathsf{negl}(\lambda),$$

where t is the time of the announcement and $\mathcal{T}_{\leq t}$ the transcript up to it.

For a Merkle tree over a random oracle the extractor recomputes the leaves from the tree of queried nodes below the root; it fails only if the adversary later opens the root to a leaf it had not queried, which requires guessing an oracle output or finding a collision. Straight-line extractability implies binding: if $\mathcal{A}$ outputs $m \neq m'$ with $\mathsf{Commit}(m) = \mathsf{Commit}(m') = \mathrm{com}$, the extractor's output on com differs from at least one of them, so the adversary $\mathcal{D}$ that runs $\mathcal{A}$, announces com and returns one of m, m' chosen at random fails extraction with at least half of $\mathcal{A}$'s success probability: $\mathsf{Adv}^{\mathsf{bind}}_{\mathsf{Com}}(\mathcal{A}) \leq 2 \cdot \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D})$.

Definition 2.3 (Preimage awareness of the accumulator). The multiset hash $\mathcal{H}$ is *preimage aware on multisets of cardinality below* 2^B [DRS09] if there is a PPT algorithm $\mathcal{E}_{\mathcal{H}}$ that, given a transcript $\mathcal{T}$ of random-oracle queries and answers and a value $y \in \mathcal{G}_{\mathcal{H}}$, outputs a multiset of typed commitments or $\perp$, such that for every PPT adversary $\mathcal{A}$ that at some point during its execution announces a value y and at the end outputs a multiset $\mathcal{S}$,

$$\mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}(\mathcal{A}) := \Pr\big[|\mathcal{S}| < 2^B \wedge \mathcal{H}(\mathcal{S}) = y \wedge \mathcal{E}_{\mathcal{H}}(\mathcal{T}_{\leq t}, y) \neq \mathcal{S}\big] \leq \mathsf{negl}(\lambda),$$

where t is the time of the announcement and $\mathcal{T}_{\leq t}$ the transcript up to it.

The definition mirrors Definition 2.2, with the accumulator in the role of the commitment: what is announced is a value of $\mathcal{G}_{\mathcal{H}}$, and what is extracted is the small multiset the adversary later opens it to. The cardinality restriction is necessary for ZisK's accumulator group: $\mathbb{Z}_p^n$ has exponent p, so adding p copies of any element to a multiset leaves $\mathcal{H}$ unchanged, and $2^B < p$ rules this out. Preimage awareness implies collision resistance on such multisets, since two distinct small multisets with the same hash would both have to equal the extractor's output.

For LtHash, collision resistance on bounded multisets reduces to the short-integer-solution problem [Ajt96, Zis25]; the reduction requires the inner digest of h to be collision resistant on its own, which a single field element is not, so we assume the digest has at least 2λ bits (Section 2.6). Preimage awareness, however, is not known to follow from any standard assumption for an algebraic accumulator, and

assuming it is the second idealization of Remark 2.1. It is nevertheless the property the analysis needs, and collision resistance alone would not do: an adversary may fix a target y by querying $\mathsf{DeriveKey}$ at it and only afterwards obtain the h-values of the elements it assembles into a multiset hashing to y, choosing which fresh values to combine after seeing them. Whether this succeeds is a preimage question, computational rather than statistical, and it is exactly what the extractor of Definition 2.3 rules out.

Definition 2.4 (Random-oracle model). The hash-to-group function h, the challenge derivation $\mathsf{DeriveKey}$ and the hash function underlying Com are independent random oracles. An adversary makes at most Q queries in total; every algorithm, including verifiers and extractors, has oracle access.

Definition 2.5 (ZisK proof system). A *ZisK proof system* is a tuple

$$\mathsf{ZisK} = \left(\mathfrak{A}, \mathsf{Com}, \mathsf{Com}_{\mathrm{cach}}, h, \mathcal{H}, \mathsf{DeriveKey}, \mathcal{F}, \left\{\Pi_{\mathrm{AIR}}^{(\tau)}\right\}_{\tau\in[M]}, \left\{\Pi_{\mathrm{LEAF}}^{(\tau)}\right\}_{\tau\in[M]}, \Pi_{\mathrm{AGG}}, \Pi_{\mathrm{FINAL}} \right)$$

consisting of the following components:

1. **AIR-type family.** $\mathfrak{A} = \{(\mathcal{C}_\tau, \mathcal{I}_\tau)\}_{\tau\in[M]}$, the constraint systems and interaction sets of Section 1.2, each with its trace height N_τ, its column classes, its prescribed data and its binding maps (Definition 1.3).
2. **Trace commitment.** Com, a deterministic commitment scheme (Definition 1.9) on witnesses of any type, straight-line extractable (Definition 2.2). It is exposed in the AIR statements (Definition 1.11), and its digests are what the leaf verifiers hash into the challenge accumulator.
3. **Cached-column commitment.** $\mathsf{Com}_{\mathrm{cach}}$, a deterministic commitment scheme on cached submatrices with commitment space $\mathbb{F}^m$. Its output travels in the public statement and is what binds a proof to a program (Definition 1.4).
4. **Hash to the accumulator group.** $h : [M] \times \{0,1\}^* \to \mathcal{G}_{\mathcal{H}}$, modelled as a random oracle (Definition 2.4), which maps a typed commitment to the leaf contribution of the challenge accumulator.
5. **Multiset hash.** $\mathcal{H}$, the additively homomorphic extension of h to finite multisets of typed commitments, preimage aware on multisets of cardinality below 2^B (Definition 2.3).
6. **Key derivation.** $\mathsf{DeriveKey} : \mathcal{G}_{\mathcal{H}} \to \mathcal{X}$, modelled as a random oracle (Definition 2.4), which turns the accumulated value into the fingerprint key that the Final Verifier checks.
7. **Bus fingerprint.** $\mathcal{F}$ (Definition 1.6), with key space $\mathcal{X}$ and values in $\mathcal{G}_{\mathcal{F}}$, statistically sound with error $\varepsilon_{\mathcal{F}}$ and admitting every nonzero multiset.
8. **Argument systems.** Each is a non-interactive argument system in the sense of Section 1.4, for the relation named.
   - $\left\{\Pi_{\mathrm{AIR}}^{(\tau)}\right\}_{\tau\in[M]}$: one per type, for $\mathcal{R}_{\mathrm{AIR}}^{(\tau)}$ (Definition 1.11), proving a single AIR instance.
   - $\left\{\Pi_{\mathrm{LEAF}}^{(\tau)}\right\}_{\tau\in[M]}$: one per type, for $\mathcal{R}_{\mathrm{LEAF}}^{(\tau)}$ (Definition 1.12), the leaf verifier, whose circuit hard-codes $\mathsf{vk}_{\mathrm{AIR}}^{(\tau)}$.
   - Π_{AGG}: for $\mathcal{R}_{\mathrm{AGG}}$ (Definition 1.14), the aggregation node, which hard-codes the M leaf-verifier keys and reads the aggregation key from its own statement.
   - Π_{FINAL}: for $\mathcal{R}_{\mathrm{FINAL}}$ (Definition 1.15), the Final Verifier, which hard-codes $\mathsf{vk}_{\mathrm{AGG}}$ and whose verifier the external party runs.

The carriers are those of the ZisK instantiation (Section 1.7), the key hard-coded by Π_{FINAL} is the key of Π_{AGG}, and the parameters satisfy the standing assumptions of Section 1.

Definition 2.6 (Bounded global satisfiability). The *target relation* $\mathcal{R}_{\mathsf{ZisK}}$ has public statements $\boldsymbol{p} \in \mathcal{P}$ and typed witness collections L as witnesses:

$$(\boldsymbol{p}; L) \in \mathcal{R}_{\mathsf{ZisK}} \Leftrightarrow \begin{cases} 1 \le |L| < 2^B, \\ \mathrm{Local}_{\tau_i}(\boldsymbol{p}, w_i) \text{ for every } i \in [|L|], \\ \mathrm{Balanced}(L). \end{cases}$$

The family $\mathfrak{A}$, the local predicates and B are implicit in the notation.

The relation is invariant under permutations of L; it requires neither that every type occur nor that witnesses be distinct. Its three conjuncts are exactly what the recursion adds to the individual AIR proofs: a bound on the number of instances, one common public statement, and global bus balancing. Everything else that a ZisK proof establishes about the guest execution is a consequence of Local and $\mathrm{Balanced}$ for the specific AIRs of the circuit and is outside this section.

Definition 2.7 (Final argument system). The *final argument system* of ZisK is the non-interactive argument system Π^* for $\mathcal{R}_{\mathsf{ZisK}}$ whose proofs are Final proofs and whose verifier is

$$V^*(\boldsymbol{p}, \pi_{\mathrm{FIN}}) := V_{\mathrm{FINAL}(\boldsymbol{p}, \pi_{\mathrm{FIN}})}.$$

Its prover is the honest proving pipeline (execute, split, prove every instance, aggregate) and plays no role below.[2]

In the reference architecture [Kad+26] the two systems differ as algorithms: its target relation has boundary machine states as statements, with memory given in full, while its last recursive proof speaks about states whose memory has been replaced by a commitment, so its external verifier must commit to the two boundary memories before it can run that proof's verifier. ZisK needs no such step: the Final relation and $\mathcal{R}_{\mathsf{ZisK}}$ have the same statement, the public statement $\boldsymbol{p}$, so Π^* and Π_{FINAL} are the same algorithm and differ only in the relation against which each is judged.

Definition 2.8 (Correct-witness extractability). ZisK is *correct-witness extractable* (CWE) if Π^* is knowledge-sound for $\mathcal{R}_{\mathsf{ZisK}}$ in the sense of Definition 2.1. We write

$$\mathsf{Adv}^{\mathsf{cwe}}_{\mathsf{ZisK}}(\mathcal{A}) := \mathsf{Adv}^{\mathsf{ks}}_{\Pi^*}(\mathcal{A})$$

for the *CWE advantage* of a PPT adversary $\mathcal{A}$ and $\mathcal{E}_{\mathsf{ZisK}}$ for the extractor whose existence is asserted; the running-time convention of Definition 2.1 applies to it unchanged.

2.3. From the Root Check to Bus Balancing.

The Final Verifier checks a single field equation, $s = 0$, on a value accumulated from the leaves under a challenge that the tree itself certifies to be derived from the leaves' commitments. By additivity of the fingerprint that value is $\mathcal{F}_\chi(\mathcal{M}_L)$ for the extracted collection L, so the check is a fingerprint zero test on the signed multiplicity of L, whereas the target relation demands $\mathcal{M}_L = 0$. This section bridges the gap. We first record that LogUp is a bus fingerprint in the sense of Definition 1.6, then that a zero test at a random challenge certifies balance, and finally that the challenge the tree certifies is random enough.

Recall ZisK's fingerprint from Section 1.7. A key is a challenge pair $\chi = (\alpha, \gamma) \in \mathbb{K}^2$, a message (b, v) is compressed to $\hat{e}_\gamma(b, v) = b + \sum_{j=1}^{\ell_b} \gamma^j v_j \in \mathbb{K}$, and

$$\mathcal{F}^{\mathrm{LogUp}}_\chi(D) = \sum_{(b,v) \in \mathrm{supp}\ D} \frac{D(b, v)}{\alpha + \hat{e}_\gamma(b, v)},$$

with $\mathcal{F}^{\mathrm{LogUp}}_\chi(D) := \perp$ if some denominator vanishes.

Lemma 2.9 (LogUp fingerprint). *$\mathcal{F}^{\mathrm{LogUp}}$ is a statistically sound bus fingerprint, with key space $\mathbb{K}^2$, values in $\mathbb{K}$ and every nonzero multiset admissible, with error*

[2] Knowledge soundness depends only on the verification algorithm.

$$\varepsilon_{\mathcal{F}}(\Lambda) = \frac{\ell \cdot \binom{\Lambda}{2} + \max(\Lambda - 1, 0)}{|\mathbb{K}|}.$$

Proof. Additivity. $\mathcal{F}_\chi^{\mathrm{LogUp}}(0) = 0$ is the empty sum. If $\mathcal{F}_\chi^{\mathrm{LogUp}}(D)$ and $\mathcal{F}_\chi^{\mathrm{LogUp}}(D')$ are both defined, no message of $\operatorname{supp} D \cup \operatorname{supp} D'$ is a pole; the support of $D + D'$ is contained in that union, and the sum defining $\mathcal{F}_\chi^{\mathrm{LogUp}}(D + D')$ splits message by message.

Soundness. If $\Lambda = 0$, there is no admissible nonzero multiset, so the advantage and the stated error are both zero. Assume $\Lambda \geq 1$. The key is drawn after the adversary has output its multiset, so it suffices to fix an arbitrary $D \neq 0$ with $|D| \leq \Lambda$ and to bound the probability over the key alone. Let $\chi = (\alpha, \gamma)$ be uniform in $\mathbb{K}^2$ and write $z_\gamma(b, v) := \hat{e}_\gamma(b, v)$ for $(b, v) \in \operatorname{supp} D$.

Compression collisions. Let E_γ be the event that $z_\gamma(b, v) = z_\gamma(b', v')$ for two distinct messages in $\operatorname{supp} D$. For a fixed pair of messages the difference is a polynomial in γ of degree at most ℓ, and it is nonzero: if $v \neq v'$ some coefficient of a positive power of γ is nonzero, and if $v = v'$ then $b \neq b'$ and the constant term is nonzero. A uniform γ is a root with probability at most $\ell/|\mathbb{K}|$, so $\Pr[E_\gamma] \leq \binom{\Lambda}{2} \cdot \ell/|\mathbb{K}|$.

Zero test. Fix γ with $\neg E_\gamma$, so the values $z_\gamma(b, v)$ are pairwise distinct, and define

$$P(X) := \sum_{(b,v) \in \operatorname{supp} D} D(b, v) \cdot \prod_{(b',v') \in \operatorname{supp} D \setminus \{(b,v)\}} \big(X + z_\gamma(b', v')\big) \in \mathbb{K}[X].$$

For any $(b_0, v_0) \in \operatorname{supp} D$,

$$P\big(-z_\gamma(b_0, v_0)\big) = D(b_0, v_0) \cdot \prod_{(b',v') \neq (b_0,v_0)} \big(z_\gamma(b', v') - z_\gamma(b_0, v_0)\big) \neq 0$$

because all factors are nonzero. Hence $P \neq 0$ and $\deg P \leq |D| - 1 \leq \Lambda - 1$. If $\mathcal{F}_\chi^{\mathrm{LogUp}}(D) = 0$, then all denominators are nonzero and multiplying through by their product gives $P(\alpha) = 0$, so α is a root of P. As α is uniform and independent of γ, this has probability at most $(\Lambda - 1)/|\mathbb{K}|$. A union bound over the two events gives the claim, and the bound is non-decreasing in Λ. □

Lemma 2.10 (Zero test). *Let $\mathcal{F}$ be a statistically sound bus fingerprint with key space $\mathcal{X}$ that admits every nonzero multiset, let L be a typed witness collection with $\neg \operatorname{Balanced}(L)$, and let χ be uniform in $\mathcal{X}$. Then*

$$\Pr_\chi\left[\sum_{i=1}^{|L|} \mathrm{BusCont}_\chi\big(\mathcal{I}_{\tau_i}, w_i\big) = 0\right] \leq \varepsilon_{\mathcal{F}}(\Lambda(L)).$$

Proof. If the sum is 0, every summand $\mathcal{F}_\chi\big(\mathrm{Msg}_{\tau_i}(w_i)\big)$ is defined, so by additivity the sum equals $\mathcal{F}_\chi\big(\sum_i \mathrm{Msg}_{\tau_i}(w_i)\big) = \mathcal{F}_\chi(\mathcal{M}_L)$. Since L is unbalanced, $\mathcal{M}_L \neq 0$, and $|\mathcal{M}_L| \leq \Lambda(L)$ (Definition 1.8), so the soundness of $\mathcal{F}$ bounds the probability by $\varepsilon_{\mathcal{F}}(\Lambda(L))$. □

The zero test is exactly where the field-valued definition of $\mathrm{Balanced}$ enters: it certifies that every signed multiplicity vanishes in $\mathbb{F}$, no more. We now define the game that captures what the root checks certify about an extracted collection, and bound it.

Definition 2.11 (Bus-argument soundness game). For an adversary $\mathcal{A}$ with access to the random oracles, the experiment $\mathsf{Exp}^{\mathsf{bus}}_{\mathcal{A}}(\lambda)$ proceeds as follows.

1. **Adversary.** $\mathcal{A}\big(1^\lambda\big)$ outputs a typed witness collection $L = ((\tau_i, w_i))_{i \in [|L|]}$ and commitments $(\mathrm{com}_i)_{i \in [|L|]}$.
2. **Derivation.** Set $\boldsymbol{c} := \sum_{i=1}^{|L|} h(\tau_i, \mathrm{com}_i) \in \mathbb{Z}_p^n$ and $\chi := \mathsf{DeriveKey}(\boldsymbol{c})$.
3. **Winning condition.** Return 1 if and only if $1 \leq |L| < 2^B$; $\mathsf{Commit}(w_i) = \mathrm{com}_i$ for every i; $\sum_{i=1}^{|L|} \mathrm{BusCont}_\chi\big(\mathcal{I}_{\tau_i}, w_i\big) = 0$; and $\neg \operatorname{Balanced}(L)$.

We write $\mathsf{Adv}^{\mathsf{bus}}(\mathcal{A}) := \Pr\big[\mathsf{Exp}^{\mathsf{bus}}_{\mathcal{A}}(\lambda) = 1\big]$.

Substituting $s_i := \mathrm{BusCont}_{\chi}\big(\mathcal{I}_{\tau_i}, w_i\big)$ and $\boldsymbol{c}_i := h(\tau_i, \mathrm{com}_i)$ shows that the winning condition is precisely "the root checks $s = 0$ and $\chi = \mathsf{DeriveKey}(\boldsymbol{c})$ pass on values honestly computed from L, yet L is unbalanced". This is the form in which the game is invoked in the proof of Theorem 2.21.

Proposition 2.12 (Bus soundness). *Let $\mathcal{F}$ be the bus fingerprint of the system. Under the idealizations of Remark 2.1, for every PPT adversary $\mathcal{A}$ making at most Q random-oracle queries there are reduction adversaries $\mathcal{D}^{\mathrm{pa}}$ and $\mathcal{D}^{\mathrm{ext}}$, each running $\mathcal{A}$ once in time $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$, such that*

$$\mathsf{Adv}^{\mathsf{bus}}(\mathcal{A}) \le (Q+1) \cdot \varepsilon_{\mathcal{F}}(\Lambda_{\max}) + \mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{pa}}) + \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D}^{\mathrm{ext}}).$$

Proof. Let $\big(L, (\mathrm{com}_i)_i\big)$ be $\mathcal{A}$'s output and $\boldsymbol{c}$, χ as in the game, and let $\mathcal{S} := \{(\tau_i, \mathrm{com}_i)\}_i$ be the multiset of typed commitments of the output, so that $\boldsymbol{c} = \mathcal{H}(\mathcal{S})$. Number the $\mathsf{DeriveKey}$ queries in the order they are made, counting only the first query at each point (a repeated query returns the answer already given), and let $t^* \in [Q+1]$ be the index of the query at $\boldsymbol{c}$, with $t^* := Q+1$ if $\mathcal{A}$ never queried $\boldsymbol{c}$; in the latter case the game's own derivation plays the role of a final, fresh query. For $t \in [Q+1]$ write $\boldsymbol{c}_t$ for the t-th query point, χ_t for its answer and $\mathcal{T}_{<t}$ for the transcript of all oracle queries and answers made before it. We define bad events, bound each by a reduction to one of the assumptions or by Lemma 2.10, and show that off all of them the adversary cannot win.

Bad events.

- E_{pa}: $|L| < 2^B$ and $\mathcal{E}_{\mathcal{H}}(\mathcal{T}_{<t^*}, \boldsymbol{c}) \neq \mathcal{S}$. The reduction $\mathcal{D}^{\mathrm{pa}}$ runs $\mathcal{A}$, announces every $\mathsf{DeriveKey}$ query point at the time of the query, and outputs $\mathcal{S}$. Its announcement at time t^* is $\boldsymbol{c}$ (if $t^* = Q+1$, the game's derivation is that announcement), so E_{pa} is its winning event of Definition 2.3 for that announcement and $\Pr\big[E_{\mathrm{pa}}\big] \le \mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{pa}})$.
- E_{ext}: $\neg E_{\mathrm{pa}}$ and $\mathcal{E}_{\mathsf{Com}}(\mathcal{T}_{<t^*}, \mathrm{com}_i) \neq w_i$ for some i. Off E_{pa} the extracted multiset at time t^* is $\mathcal{S}$, so this is the failure event of Definition 2.2 for the adversary $\mathcal{D}^{\mathrm{ext}}$ that runs $\mathcal{A}$, at every $\mathsf{DeriveKey}$ query t announces the commitments of $\mathcal{E}_{\mathcal{H}}(\mathcal{T}_{<t}, \boldsymbol{c}_t)$, and outputs $(\mathrm{com}_i, w_i)_i$; its announcement at time t^* contains every com_i. Hence $\Pr[E_{\mathrm{ext}}] \le \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D}^{\mathrm{ext}})$.
- $E^{(t)}_{\chi}$ for $t \in [Q+1]$: define the *candidate collection* L^*_t as follows. If $\mathcal{E}_{\mathcal{H}}(\mathcal{T}_{<t}, \boldsymbol{c}_t)$ returns a multiset $\mathcal{S}_t$, let $L^*_t := \big((\tau, \mathcal{E}_{\mathsf{Com}}(\mathcal{T}_{<t}, \mathrm{com}))\big)_{(\tau,\, \mathrm{com}) \in \mathcal{S}_t}$ in a canonical order, discarding the collection if some extraction returns $\perp$; otherwise $L^*_t := \perp$. The event $E^{(t)}_{\chi}$ is that $L^*_t \neq \perp$, $|L^*_t| < 2^B$, $\neg\ \mathrm{Balanced}(L^*_t)$ and $\sum_i \mathrm{BusCont}_{\chi_t}\big(\mathcal{I}_{\tau_i}, w_i\big) = 0$ over the entries of L^*_t. The candidate is a function of $\mathcal{T}_{<t}$ alone, whereas χ_t is uniform in $\mathbb{K}^2$ and independent of $\mathcal{T}_{<t}$ because t is the first query at its point; since $|L^*_t| < 2^B$ implies $\Lambda(L^*_t) \le \Lambda_{\max}$, Lemma 2.10 gives $\Pr\big[E^{(t)}_{\chi}\big] \le \varepsilon_{\mathcal{F}}(\Lambda_{\max})$.

Step A: off all events, the adversary does not win. Suppose $\mathcal{A}$ wins and none of E_{pa}, E_{ext}, $E^{(t^*)}_{\chi}$ occurs. The win gives $|L| < 2^B$, so off E_{pa}, $\mathcal{E}_{\mathcal{H}}(\mathcal{T}_{<t^*}, \boldsymbol{c}_{t^*}) = \mathcal{S}$. Off E_{ext}, $\mathcal{E}_{\mathsf{Com}}(\mathcal{T}_{<t^*}, \mathrm{com}_i) = w_i$ for every i, so $L^*_{t^*}$ is a reordering of L; every quantity in the winning condition is invariant under reordering. The winning condition then says $\neg\ \mathrm{Balanced}(L^*_{t^*})$ and $\sum_i \mathrm{BusCont}_{\chi_{t^*}}\big(\mathcal{I}_{\tau_i}, w_i\big) = 0$ with $\chi = \chi_{t^*}$, which is $E^{(t^*)}_{\chi}$: a contradiction.

Conclusion. By Step A a win implies one of the bad events, and a union bound over E_{pa}, E_{ext} and $E^{(1)}_{\chi}, \ldots, E^{(Q+1)}_{\chi}$ gives the stated sum. Both reductions run $\mathcal{A}$ once and perform a scan of the transcript, so they run in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. □

Remark 2.3 (Removing the idealization). *Without preimage awareness of $\mathcal{H}$, the second idealization of Remark 2.1, the event E_{pa} must be handled by rewinding, using only the collision resistance of $\mathcal{H}$ on multisets of cardinality below 2^B, with advantage $\mathsf{Adv}^{\mathsf{cr}}_{\mathcal{H}}(\mathcal{A}) := \Pr\big[(\mathcal{S}, \mathcal{S}') \leftarrow \mathcal{A} : \mathcal{S} \neq \mathcal{S}' \wedge |\mathcal{S}|, |\mathcal{S}'| < 2^B \wedge \mathcal{H}(\mathcal{S}) = \mathcal{H}(\mathcal{S}')\big]$, which for LtHash reduces to the short-integer-solution problem. Run $\mathcal{A}$ a second time from its state just before the query t^*, answering it with a fresh χ'. If both runs win with the same accumulator and the output collections differ, the two multisets (or two openings of one commitment) form a collision for $\mathcal{H}$ or a break of Definition 2.2; if they coincide, a single unbalanced collection has passed the zero test*

at two independent challenges, which Lemma 2.10 bounds. The general forking lemma [BN06] then yields, schematically,

$$\mathsf{Adv}^{\mathsf{bus}}(\mathcal{A}) \le \varepsilon_{\mathcal{F}}(\Lambda_{\max}) + \sqrt{(Q+1)\big(\varepsilon_{\mathcal{F}}(\Lambda_{\max}) + \mathsf{Adv}^{\mathsf{cr}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{cr}}) + \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D}^{\mathrm{ext}})\big)} + \frac{Q+1}{|\mathbb{K}|},$$

with reductions that run $\mathcal{A}$ twice. The square-root loss is inherent to rewinding at a Fiat–Shamir query [FS87] and is far from concretely tight. Making the linear bound of Proposition 2.12 rigorous is a proof obligation for the full security argument: either preimage awareness of LtHash as a knowledge-type assumption (an efficient extractor that recovers the multiset from an accumulator the adversary produced), or an accumulator that is preimage aware in the ROM by construction, at the cost of the homomorphism that makes independent aggregation possible.

Remark 2.4 (Field versus integer balance). *Proposition 2.12 delivers* $\mathrm{Balanced}(L)$ *with multiplicities in* $\mathbb{F}$*. To conclude that every message is sent and received the same number of times as integers one needs, in addition, that every multiplicity column of every AIR is range-checked to* $[0, m_{\max}]$ *(or to a symmetric interval with a fixed signed encoding) and that* $\Lambda_{\max} \cdot m_{\max} < p$*, so that vanishing in* $\mathbb{F}$ *lifts to vanishing in* $\mathbb{Z}$*. Whether the receiving sides of ZisK's lookup tables satisfy such a bound is a property of the individual AIRs and is recorded as an open item in Section 2.6.*

2.4. Lemmas: Descending the Aggregation Tree.

We state one extraction game and one lemma per layer of the aggregation tree, proceeding from the Final Verifier inward to the AIR arguments. Each lemma composes the straight-line extractors guaranteed by Definition 2.1.

Advantage notation. We collect the advantages introduced above: $\mathsf{Adv}^{\mathsf{ks}}_{\Pi}$ (knowledge-soundness error, Definition 2.1), $\mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}$ (Definition 2.2), $\mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}$ (Definition 2.3) and $\mathsf{Adv}^{\mathsf{bus}}$ (Definition 2.11). All advantages are functions of λ. In each lemma we exhibit explicit reduction adversaries $\mathcal{D}$ and state their running times; when a bound has the form $k \cdot \mathsf{Adv}^{\cdot}(\mathcal{D})$, the reduction $\mathcal{D}$ denotes the maximum-advantage adversary among the k per-node reductions constructed in the proof, each of which runs in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. Probabilities in the games below are over the coins of the adversary, the extractor and the random oracles; verification is deterministic.

2.4.1. *Final Layer.*

Definition 2.13 (Final extraction game). For an extractor $\mathcal{E}_{\mathrm{FIN}}$ and a PPT adversary $\mathcal{A}$, the experiment $\mathsf{Exp}^{\mathsf{fin}}_{(\mathcal{E}_{\mathrm{FIN}},\mathcal{A})}(\lambda)$ is:

1. **Adversary.** $\mathcal{A}(1^\lambda)$ outputs a public statement $\boldsymbol{p} \in \mathcal{P}$ and a proof π_{FIN}.
2. **Filter.** If $V_{\mathrm{FINAL}(\boldsymbol{p},\pi_{\mathrm{FIN}})} \neq 1$, return 0.
3. **Extraction.** Run $(\chi, \sigma, \pi) \leftarrow \mathcal{E}_{\mathrm{FIN}(\boldsymbol{p},\pi_{\mathrm{FIN}})}$, with $\sigma = (\tau, \kappa, \boldsymbol{c}, s)$.
4. **Winning condition.** Return 1 if and only if $(\boldsymbol{p}; (\chi, \sigma, \pi)) \notin \mathcal{R}_{\mathrm{FINAL}}$, i.e. unless $\mathsf{Typed}(\tau, \kappa)$, $V^{(\tau)}_{\mathrm{REC}}[\mathsf{vk}_{\mathrm{AGG}}]((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}_{\mathrm{AGG}}), \pi) = 1$, $s = 0$ and $\chi = \mathsf{DeriveKey}(\boldsymbol{c})$.

We write $\mathsf{Adv}^{\mathsf{fin}}_{\mathcal{E}_{\mathrm{FIN}}}(\mathcal{A})$ for the probability that the experiment $\mathsf{Exp}^{\mathsf{fin}}_{(\mathcal{E}_{\mathrm{FIN}},\mathcal{A})}(\lambda)$ returns 1.

Lemma 2.14 (Final extraction). *Assume* Π_{FINAL} *is knowledge-sound and let* $\mathcal{E}_{\mathrm{FIN}} := \mathcal{E}_{\Pi_{\mathrm{FINAL}}}$ *be its extractor. For every PPT adversary* $\mathcal{A}$ *there is a reduction* $\mathcal{D}_{\mathrm{FIN}}$ *with* $\mathsf{Time}(\mathcal{D}_{\mathrm{FIN}}) = \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ *and*

$$\mathsf{Adv}^{\mathsf{fin}}_{\mathcal{E}_{\mathrm{FIN}}}(\mathcal{A}) = \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{FINAL}}}(\mathcal{D}_{\mathrm{FIN}}).$$

Proof. $\mathcal{D}_{\mathrm{FIN}}$ runs $\mathcal{A}(1^\lambda)$ and forwards $(\boldsymbol{p}, \pi_{\mathrm{FIN}})$ to the knowledge-soundness challenger of Π_{FINAL}. The challenger applies the same filter and runs the same extractor, and Definition 2.13 returns 1 exactly when π_{FIN} verifies but $(\boldsymbol{p}; (\chi, \sigma, \pi)) \notin \mathcal{R}_{\mathrm{FINAL}}$, which is the event that $\mathcal{D}_{\mathrm{FIN}}$ wins. Since Π^* and Π_{FINAL} are the same algorithm, this is the base of the composition and no additional work is required. □

2.4.2. *Aggregation Layer.*

The aggregation extractor unrolls the tree below the Final Verifier's child until it reaches leaf-verifier proofs. The counter κ of the child statement tells it in advance how many leaves to expect, and it guarantees that it never has to descend further than $\kappa - 1$ aggregation nodes.

Definition 2.15 (Aggregation extraction game). For a counter $\kappa \in [1, 2^B)$, an extractor $\mathcal{E}_{\mathrm{REC}}$ and a PPT adversary $\mathcal{A}$, the experiment $\mathsf{Exp}^{\mathsf{agg}}_{(\kappa,\mathcal{E}_{\mathrm{REC}},\mathcal{A})}(\lambda)$ is:

1. **Adversary.** $\mathcal{A}(1^\lambda)$ outputs $(\boldsymbol{p}, \chi, \sigma, \pi)$ with $\sigma = (\tau, \kappa, \boldsymbol{c}, s)$.
2. **Filter.** If $\neg\ \mathsf{Typed}(\tau, \kappa)$ or $V^{(\tau)}_{\mathrm{REC}}[\mathsf{vk}_{\mathrm{AGG}}]((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}_{\mathrm{AGG}}), \pi) \neq 1$, return 0.
3. **Extraction.** Run $((\sigma_1, \pi_1), ..., (\sigma_\kappa, \pi_\kappa)) \leftarrow \mathcal{E}_{\mathrm{REC}(\boldsymbol{p},\chi,\sigma,\pi)}$, with $\sigma_i = (\tau_i, \kappa_i, \boldsymbol{c}_i, s_i)$.
4. **Winning condition.** Return 1 unless, for every $i \in [\kappa]$, $\tau_i \in [M]$, $\kappa_i = 1$ and $V^{(\tau_i)}_{\mathrm{LEAF}}((\boldsymbol{p}, \chi, \sigma_i, \mathsf{vk}_{\mathrm{AGG}}), \pi_i) = 1$, and moreover $\sum_{i=1}^{\kappa} s_i = s$ and $\sum_{i=1}^{\kappa} \boldsymbol{c}_i = \boldsymbol{c}$.

We write $\mathsf{Adv}^{\mathsf{agg}}_{(\kappa,\mathcal{E}_{\mathrm{REC}})}(\mathcal{A})$ for the probability that the experiment returns 1.

Lemma 2.16 (Aggregation tree unrolling). *Assume Π_{AGG} is knowledge-sound and let $\kappa \in [1, 2^B)$. There is an extractor $\mathcal{E}_{\mathrm{REC}}$, given in the proof, such that for every PPT adversary $\mathcal{A}$ there is a reduction $\mathcal{D}_{\mathrm{AGG}}$ running in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ with*

$$\mathsf{Adv}^{\mathsf{agg}}_{(\kappa,\mathcal{E}_{\mathrm{REC}})}(\mathcal{A}) \leq (\kappa - 1) \cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}(\mathcal{D}_{\mathrm{AGG}}).$$

$\mathcal{E}_{\mathrm{REC}}$ makes at most $\kappa - 1$ calls to $\mathcal{E}_{\Pi_{\mathrm{AGG}}}$ and runs in $(\kappa - 1) \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$.

Proof. The game returns 0 unless the filter passes, so we assume it does and bound the probability that extraction violates the winning condition. The argument runs in four steps.

Step 1: recursion tree and bad events. $\mathcal{E}_{\mathrm{REC}}$ unrolls a binary tree depth-first. A node v is a pair (x_v, π_v) with $x_v = (\boldsymbol{p}, \chi, \sigma_v, \mathsf{vk}_{\mathrm{AGG}})$; the root is $\mathcal{A}$'s output. If $\sigma_v.\tau \in [M]$, the node is a *leaf* and the extractor records (σ_v, π_v). If $\sigma_v.\tau = M + 1$, the extractor runs $\mathcal{E}_{\Pi_{\mathrm{AGG}}}(x_v, \pi_v)$ to obtain a candidate witness $((\sigma_L, \pi_L), (\sigma_R, \pi_R))$ and recurses into the left child $((\boldsymbol{p}, \chi, \sigma_L, \mathsf{vk}_{\mathrm{AGG}}), \pi_L)$ and then the right child; if the candidate is not a valid $\mathcal{R}_{\mathrm{AGG}}$-witness, the extractor aborts. For $t \geq 1$ let

$$B_t : \quad \text{the first } \ \mathcal{E}_{\Pi_{\mathrm{AGG}}} \ \text{ invocation in unroll order that does not return}$$
$$\text{a valid } \ \mathcal{R}_{\mathrm{AGG}}\text{-witness for its node's statement is invocation } \ t\ .$$

The events are disjoint, and at every invocation the node's proof verifies under $V(\mathsf{vk}_{\mathrm{AGG}}, \cdot, \cdot)$: the root's by the filter (since $\mathsf{Typed}(\tau, \kappa)$ with $\tau = M + 1$ and $V^{(M+1)}_{\mathrm{REC}}[\mathsf{vk}_{\mathrm{AGG}}] = V(\mathsf{vk}_{\mathrm{AGG}}, \cdot, \cdot)$), a child's by its parent's valid witness. Now count. Before the first failure, the $t - 1$ valid aggregation nodes form a binary tree whose frontier consists of t nodes, each not yet expanded; by the counter invariant every frontier node carries an integer counter at least 1, and the counters of the frontier sum to κ. Hence $t \leq \kappa$. Moreover, a frontier node of type $M + 1$ has counter at least 2 by Typed, so if $\kappa - 1$ valid nodes exist the frontier has κ nodes with counter exactly 1, all of type in $[M]$, and no further invocation exists. Only $B_1, ..., B_{\kappa-1}$ are possible, and off all of them the tree has exactly $\kappa - 1$ aggregation nodes and κ leaves; for $\kappa = 1$ the root is a leaf and there is nothing to extract.

Step 2: one reduction per invocation. $\mathcal{D}^{(t)}_{\mathrm{AGG}}$ runs $\mathcal{A}(1^\lambda)$, unrolls the tree up to the t-th invocation (which is deterministic given $\mathcal{A}$'s output and the earlier extractions) and forwards that node's (x_v, π_v) to the knowledge-soundness challenger of Π_{AGG}. It wins whenever B_t occurs, so $\Pr[B_t] \leq \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}\left(\mathcal{D}^{(t)}_{\mathrm{AGG}}\right)$, and it runs in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$.

Step 3: off the bad events, the extractor succeeds. Assume $\neg B_1 \wedge ... \wedge \neg B_{\kappa-1}$; every statement below is then deterministic. We show by strong induction on the counter κ_v that for every node v of the unrolled tree, the leaves recorded below v number exactly κ_v, each has a type in $[M]$ and counter 1, each verifies

under $V_{\mathrm{LEAF}}^{(\tau_i)}$ on the statement $(\boldsymbol{p}, \chi, \sigma_i, \mathsf{vk}_{\mathrm{AGG}})$, and their partial sums and challenge contributions add up to $\sigma_v.s$ and $\sigma_v.\boldsymbol{c}$.

- *Leaf nodes.* If $\sigma_v.\tau \in [M]$, then Typed (asserted by the parent's valid witness, or by the filter for the root) gives $\kappa_v = 1$, and the same source gives $V_{\mathrm{REC}}^{(\tau_v)}[\mathsf{vk}_{\mathrm{AGG}}](x_v, \pi_v) = V_{\mathrm{LEAF}}^{(\tau_v)}(x_v, \pi_v) = 1$. The sums are trivial.
- *Aggregation nodes.* If $\sigma_v.\tau = M + 1$, the valid witness returned for v provides typed children with $\kappa_L + \kappa_R = \kappa_v$ over the integers and $\kappa_X \in [1, \kappa_v - 1]$, each verifying under $V_{\mathrm{REC}}^{(\tau_X)}[\mathsf{vk}_{\mathrm{AGG}}]$ on a statement that, by the propagation invariant, carries the same $(\boldsymbol{p}, \chi, \mathsf{vk}_{\mathrm{AGG}})$; a child of type $M + 1$ is therefore verified by $V(\mathsf{vk}_{\mathrm{AGG}}, \cdot, \cdot) = V_{\mathrm{AGG}}$ and is itself an aggregation node of the tree. The induction hypothesis applies to both children; concatenating their leaf lists gives $\kappa_L + \kappa_R = \kappa_v$ leaves, and $s_v = s_L + s_R$, $\boldsymbol{c}_v = \boldsymbol{c}_L + \boldsymbol{c}_R$ make the sums telescope.

Applied to the root, this is exactly the winning condition of Definition 2.15.

Step 4: conclusion. Off all bad events the extractor meets the winning condition, so the game returns 1 only if some B_t occurs, and only $B_1, \ldots, B_{\kappa-1}$ are possible. By a union bound

$$\mathsf{Adv}^{\mathsf{agg}}_{(\kappa, \mathcal{E}_{\mathrm{REC}})}(\mathcal{A}) \le \sum_{t=1}^{\kappa-1} \Pr[B_t] \le (\kappa - 1) \cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}(\mathcal{D}_{\mathrm{AGG}}),$$

with $\mathcal{D}_{\mathrm{AGG}}$ the maximum-advantage reduction among the $\mathcal{D}_{\mathrm{AGG}}^{(t)}$. The extractor invokes $\mathcal{E}_{\Pi_{\mathrm{AGG}}}$ once per aggregation node, at most $\kappa - 1$ times, each call costing at most $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. □

The proof relies on the typing predicate at three places where a proof could *verify* for a statement outside the relation's support, which knowledge soundness excludes only up to its error probability: a child of type $M + 1$ with counter 1 (whose statement has no witness), a child of type in $[M]$ with a counter other than 1 (outside the support of the leaf relation), and a child type outside $[M + 1]$ (for which no verifier is selected). In each case the parent's valid witness excludes the child, so the offending proof is charged to the parent's invocation.

2.4.3. *Leaf-Verifier Layer.*

Definition 2.17 (Leaf-verifier extraction game). For $\tau \in [M]$, an extractor $\mathcal{E}_{\mathrm{LEAF}}^{(\tau)}$ and a PPT adversary $\mathcal{A}$, the experiment $\mathsf{Exp}^{\mathsf{leaf}}_{\left(\tau, \mathcal{E}_{\mathrm{LEAF}}^{(\tau)}, \mathcal{A}\right)}(\lambda)$ is:

1. **Adversary.** $\mathcal{A}\big(1^\lambda\big)$ outputs $(\boldsymbol{p}, \chi, \sigma, \mathsf{vk})$ with $\sigma = (\tau', \kappa, \boldsymbol{c}, s)$, and a proof π_{LEAF}.
2. **Filter.** If $V_{\mathrm{LEAF}}^{(\tau)}((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}), \pi_{\mathrm{LEAF}}) \neq 1$, return 0.
3. **Extraction.** Run $(\mathrm{com}, \pi_{\mathrm{AIR}}) \leftarrow \mathcal{E}_{\mathrm{LEAF}}^{(\tau)}((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}), \pi_{\mathrm{LEAF}})$.
4. **Winning condition.** Return 1 if and only if $((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}); (\mathrm{com}, \pi_{\mathrm{AIR}})) \notin \mathcal{R}_{\mathrm{LEAF}}^{(\tau)}$, i.e. unless $\tau' = \tau$, $\kappa = 1$, $V_{\mathrm{AIR}}^{(\tau)}((\boldsymbol{p}, \chi, s, \mathrm{com}), \pi_{\mathrm{AIR}}) = 1$ and $\boldsymbol{c} = h(\tau, \mathrm{com})$.

We write $\mathsf{Adv}^{\mathsf{leaf}}_{\left(\tau, \mathcal{E}_{\mathrm{LEAF}}^{(\tau)}\right)}(\mathcal{A})$ for the probability that the experiment returns 1.

Lemma 2.18 (Leaf-verifier extraction). *Assume $\Pi_{\mathrm{LEAF}}^{(\tau)}$ is knowledge-sound and let $\mathcal{E}_{\mathrm{LEAF}}^{(\tau)} := \mathcal{E}_{\Pi_{\mathrm{LEAF}}^{(\tau)}}$. For every PPT adversary $\mathcal{A}$,*

$$\mathsf{Adv}^{\mathsf{leaf}}_{\left(\tau, \mathcal{E}_{\mathrm{LEAF}}^{(\tau)}\right)}(\mathcal{A}) = \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{LEAF}}^{(\tau)}}\left(\mathcal{D}_{\mathrm{LEAF}}^{(\tau)}\right), \quad \mathcal{D}_{\mathrm{LEAF}}^{(\tau)} := \mathcal{A}.$$

Proof. The experiment is the knowledge-soundness game of $\Pi_{\mathrm{LEAF}}^{(\tau)}$: same filter, same extractor, and the winning condition is non-membership of the extracted pair in $\mathcal{R}_{\mathrm{LEAF}}^{(\tau)}$. A statement with $\tau' \neq \tau$ or $\kappa \neq 1$ lies outside the support of the relation; a verifying proof for it is a knowledge-soundness break, and the experiment returns 1 for it as well, so the two advantages coincide. The key vk is not constrained at this layer and is only forwarded. □

The type under which a leaf is extracted is the type $\sigma_i.\tau$ read from its statement, which is also the type that selected the verifier in the aggregation layer. The extracted AIR proof is therefore a proof for $\mathcal{C}_{\sigma_i.\tau}$, and this is what makes the output of the composed extractor a *typed* collection.

2.4.4. *AIR Layer.*

Definition 2.19 (AIR extraction game). For $\tau \in [M]$, an extractor $\mathcal{E}_{\mathrm{AIR}}^{(\tau)}$ and a PPT adversary $\mathcal{A}$, the experiment $\mathsf{Exp}^{\mathsf{air}}_{\left(\tau,\mathcal{E}_{\mathrm{AIR}}^{(\tau)},\mathcal{A}\right)}(\lambda)$ is:

1. **Adversary.** $\mathcal{A}\big(1^\lambda\big)$ outputs a statement $x = (\boldsymbol{p}, \chi, s, \mathrm{com})$ and a proof π_{AIR}.
2. **Filter.** If $V_{\mathrm{AIR}}^{(\tau)}(x, \pi_{\mathrm{AIR}}) \neq 1$, return 0.
3. **Extraction.** Run $(\boldsymbol{T}, \boldsymbol{a}) \leftarrow \mathcal{E}_{\mathrm{AIR}}^{(\tau)}(x, \pi_{\mathrm{AIR}})$.
4. **Winning condition.** Return 1 if and only if $(x; (\boldsymbol{T}, \boldsymbol{a})) \notin \mathcal{R}_{\mathrm{AIR}}^{(\tau)}$, i.e. unless $\mathrm{Local}_\tau(\boldsymbol{p}, (\boldsymbol{T}, \boldsymbol{a}))$, $s = \mathrm{BusCont}_\chi(\mathcal{I}_\tau, (\boldsymbol{T}, \boldsymbol{a}))$ and $\mathsf{Commit}(\boldsymbol{T}, \boldsymbol{a}) = \mathrm{com}$.

We write $\mathsf{Adv}^{\mathsf{air}}_{\left(\tau,\mathcal{E}_{\mathrm{AIR}}^{(\tau)}\right)}(\mathcal{A})$ for the probability that the experiment returns 1.

Lemma 2.20 (AIR extraction). *Assume $\Pi_{\mathrm{AIR}}^{(\tau)}$ is knowledge-sound and let $\mathcal{E}_{\mathrm{AIR}}^{(\tau)} := \mathcal{E}_{\Pi_{\mathrm{AIR}}^{(\tau)}}$. For every PPT adversary $\mathcal{A}$,*

$$\mathsf{Adv}^{\mathsf{air}}_{\left(\tau,\mathcal{E}_{\mathrm{AIR}}^{(\tau)}\right)}(\mathcal{A}) = \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AIR}}^{(\tau)}}\left(\mathcal{D}_{\mathrm{AIR}}^{(\tau)}\right), \quad \mathcal{D}_{\mathrm{AIR}}^{(\tau)} := \mathcal{A},$$

and whenever the experiment returns 0 after passing the filter, the extracted witness $(\boldsymbol{T}, \boldsymbol{a}) \in \mathcal{W}_\tau$ satisfies $\mathrm{Local}_\tau(\boldsymbol{p}, (\boldsymbol{T}, \boldsymbol{a}))$, has bus contribution s under χ, and commits to com.

Proof. The experiment is the knowledge-soundness game of $\Pi_{\mathrm{AIR}}^{(\tau)}$, and the structure of the extracted witness is the definition of $\mathcal{R}_{\mathrm{AIR}}^{(\tau)}$. □

2.5. **Main Theorem.**

Theorem 2.21 (Correct-witness extractability). *Let* ZisK *be a ZisK proof system (Definition 2.5) and let $\kappa_{\max} := 2^B - 1$. Under the idealizations of Remark 2.1, for every PPT adversary $\mathcal{A}$ making at most Q random-oracle queries, with CWE advantage $\varepsilon := \mathsf{Adv}^{\mathsf{cwe}}_{\mathsf{ZisK}}(\mathcal{A})$, there exist PPT reduction adversaries $\mathcal{D}_{\mathrm{FIN}}$, $\mathcal{D}_{\mathrm{AGG}}$, $\left\{\mathcal{D}_{\mathrm{LEAF}}^{(\tau)}\right\}_\tau$, $\left\{\mathcal{D}_{\mathrm{AIR}}^{(\tau)}\right\}_\tau$, $\mathcal{D}^{\mathrm{pa}}$ and $\mathcal{D}^{\mathrm{ext}}$ such that*

$$\begin{aligned}\varepsilon \le\ & \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{FINAL}}}(\mathcal{D}_{\mathrm{FIN}}) + (\kappa_{\max} - 1) \cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}(\mathcal{D}_{\mathrm{AGG}}) \\ & + \kappa_{\max} \cdot \max_{\tau \in [M]} \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{LEAF}}^{(\tau)}}\left(\mathcal{D}_{\mathrm{LEAF}}^{(\tau)}\right) + \kappa_{\max} \cdot \max_{\tau \in [M]} \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AIR}}^{(\tau)}}\left(\mathcal{D}_{\mathrm{AIR}}^{(\tau)}\right) \\ & + (Q+1) \cdot \varepsilon_{\mathcal{F}}(\Lambda_{\max}) + \mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{pa}}) + \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D}^{\mathrm{ext}}).\end{aligned}$$

All reduction adversaries invoke $\mathcal{A}$ exactly once and run in time at most $(3\kappa_{\max} + 1) \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. In particular, if every advantage on the right-hand side is negligible in λ, then ZisK *is correct-witness extractable (Definition 2.8).*

Proof. Let $\mathcal{A}$ be a PPT adversary with CWE advantage ε; we bound ε. We construct $\mathcal{E}_{\mathsf{ZisK}}$ by composing the sub-extractors of Lemma 2.14, Lemma 2.16, Lemma 2.18 and Lemma 2.20, each straight-line and each applied to the values extracted by the preceding layer. Let $(\boldsymbol{p}, \pi_{\mathrm{FIN}})$ be $\mathcal{A}$'s output and assume $V_{\mathrm{FINAL}(\boldsymbol{p}, \pi_{\mathrm{FIN}})} = 1$, since otherwise $\mathcal{A}$ does not win.

Step 1 (Final layer). Apply $\mathcal{E}_{\mathrm{FIN}}$ to $(\boldsymbol{p}, \pi_{\mathrm{FIN}})$ to obtain (χ, σ, π) with $\sigma = (\tau, \kappa, \boldsymbol{c}, s)$ such that $\mathsf{Typed}(\tau, \kappa)$, $V_{\mathrm{REC}}^{(\tau)}[\mathsf{vk}_{\mathrm{AGG}}]((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}_{\mathrm{AGG}}), \pi) = 1$, $s = 0$ and $\chi = \mathsf{DeriveKey}(\boldsymbol{c})$. This fails with probability at most $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{FINAL}}}(\mathcal{D}_{\mathrm{FIN}})$ by Lemma 2.14. In particular $\kappa \in [1, 2^B)$, so $\kappa \le \kappa_{\max}$.[3]

Step 2 (Aggregation layer). Apply $\mathcal{E}_{\mathrm{REC}}$ to $(\boldsymbol{p}, \chi, \sigma, \pi)$; its filter is satisfied by Step 1. Obtain $((\sigma_1, \pi_1), ..., (\sigma_\kappa, \pi_\kappa))$ with $\sigma_i = (\tau_i, 1, \boldsymbol{c}_i, s_i)$, $\tau_i \in [M]$, $V_{\mathrm{LEAF}}^{(\tau_i)}((\boldsymbol{p}, \chi, \sigma_i, \mathsf{vk}_{\mathrm{AGG}}), \pi_i) = 1$, $\sum_i s_i = s = 0$ and $\sum_i \boldsymbol{c}_i = \boldsymbol{c}$. This fails with probability at most $(\kappa - 1) \cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}(\mathcal{D}_{\mathrm{AGG}})$ by Lemma 2.16, and the extractor runs in $(\kappa - 1) \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$.

Step 3 (Leaf-verifier layer, applied κ times). For each $i \in [\kappa]$ apply $\mathcal{E}_{\mathrm{LEAF}}^{(\tau_i)}$ to $((\boldsymbol{p}, \chi, \sigma_i, \mathsf{vk}_{\mathrm{AGG}}), \pi_i)$ to obtain $(\mathrm{com}_i, \pi_i^{\mathrm{AIR}})$ with $V_{\mathrm{AIR}}^{(\tau_i)}((\boldsymbol{p}, \chi, s_i, \mathrm{com}_i), \pi_i^{\mathrm{AIR}}) = 1$ and $\boldsymbol{c}_i = h(\tau_i, \mathrm{com}_i)$. Each invocation fails with probability at most $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{LEAF}}^{(\tau_i)}}\left(\mathcal{D}_{\mathrm{LEAF}}^{(\tau_i)}\right)$ by Lemma 2.18; the total over κ calls is at most $\kappa \cdot \max_\tau \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{LEAF}}^{(\tau)}}\left(\mathcal{D}_{\mathrm{LEAF}}^{(\tau)}\right)$.

Step 4 (AIR layer, applied κ times). For each $i \in [\kappa]$ apply $\mathcal{E}_{\mathrm{AIR}}^{(\tau_i)}$ to $\left((\boldsymbol{p}, \chi, s_i, \mathrm{com}_i), \pi_i^{\mathrm{AIR}}\right)$ to obtain $w_i = (\boldsymbol{T}_i, \boldsymbol{a}_i) \in \mathcal{W}_{\tau_i}$ with $\mathrm{Local}_{\tau_i}(\boldsymbol{p}, w_i)$, $s_i = \mathrm{BusCont}_\chi\left(\mathcal{I}_{\tau_i}, w_i\right)$ and $\mathsf{Commit}(w_i) = \mathrm{com}_i$. The total failure probability over κ calls is at most $\kappa \cdot \max_\tau \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AIR}}^{(\tau)}}\left(\mathcal{D}_{\mathrm{AIR}}^{(\tau)}\right)$ by Lemma 2.20.

Step 5 (Assembly). Output $L := ((\tau_1, w_1), ..., (\tau_\kappa, w_\kappa))$. Deterministically, given that Steps 1 to 4 succeeded: $|L| = \kappa \in [1, 2^B)$; $\mathrm{Local}_{\tau_i}(\boldsymbol{p}, w_i)$ for every i; $\boldsymbol{c} = \sum_i h(\tau_i, \mathrm{com}_i) = \sum_i h(\tau_i, \mathsf{Commit}(w_i))$; $\chi = \mathsf{DeriveKey}(\boldsymbol{c})$; and $\sum_i \mathrm{BusCont}_\chi\left(\mathcal{I}_{\tau_i}, w_i\right) = \sum_i s_i = s = 0$.

Step 6 (Bus soundness). Let $\mathcal{A}_{\mathrm{bus}}$ be the algorithm that runs $\mathcal{A}$ and Steps 1 to 5 and outputs $\left(L, (\mathrm{com}_i)_i\right)$; it makes the same oracle queries as $\mathcal{A}$ plus those of the extractors. When Steps 1 to 4 succeed, Step 5 shows that the output satisfies every conjunct of the winning condition of Definition 2.11 except possibly $\neg\ \mathrm{Balanced}(L)$, and the game's derivation reproduces the extracted $\boldsymbol{c}$ and χ. Hence

$$\begin{aligned}\Pr[\text{Steps 1 to 4 succeed} \wedge \neg\ \mathrm{Balanced}(L)] &\le \mathsf{Adv}^{\mathsf{bus}}(\mathcal{A}_{\mathrm{bus}}) \\ &\le (Q+1) \cdot \varepsilon_{\mathcal{F}}(\Lambda_{\max}) + \mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{pa}}) + \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D}^{\mathrm{ext}})\end{aligned}$$

by Proposition 2.12, where $\mathcal{D}^{\mathrm{pa}}$ and $\mathcal{D}^{\mathrm{ext}}$ run $\mathcal{A}_{\mathrm{bus}}$ once.

Conclusion. If Steps 1 to 4 succeed and L is balanced, then $(\boldsymbol{p}; L) \in \mathcal{R}_{\mathsf{ZisK}}$ by Step 5 and Definition 2.6, so $\mathcal{A}$ does not win. Summing the failure probabilities,

$$\begin{aligned}\varepsilon \le\ & \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{FINAL}}}(\mathcal{D}_{\mathrm{FIN}}) && \text{(Step 1)} \\ &+(\kappa - 1) \cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}(\mathcal{D}_{\mathrm{AGG}}) && \text{(Step 2)} \\ &+\kappa \cdot \max_\tau \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{LEAF}}^{(\tau)}}\left(\mathcal{D}_{\mathrm{LEAF}}^{(\tau)}\right) && \text{(Step 3)} \\ &+\kappa \cdot \max_\tau \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AIR}}^{(\tau)}}\left(\mathcal{D}_{\mathrm{AIR}}^{(\tau)}\right) && \text{(Step 4)} \\ &+\mathsf{Adv}^{\mathsf{bus}}(\mathcal{A}_{\mathrm{bus}}) && \text{(Step 6),}\end{aligned}$$

and replacing κ by $\kappa_{\max}$, which only increases the right-hand side, gives the stated bound. All coefficients are polynomially bounded, so the right-hand side is negligible whenever each advantage is. Unlike the step count of the reference architecture [Kad+26], κ is not a system parameter but part of the extracted witness; the bound with κ is therefore a per-tree refinement and the bound with $\kappa_{\max}$ the unconditional one.

Running time. $\mathcal{E}_{\mathsf{ZisK}}$ never runs $\mathcal{A}$; its cost is the straight-line sub-extractor calls plus deterministic work: one call to $\mathcal{E}_{\Pi_{\mathrm{FINAL}}}$, at most $\kappa - 1$ calls to $\mathcal{E}_{\Pi_{\mathrm{AGG}}}$, and κ calls each to the leaf-verifier and AIR extractors, i.e. at most 3κ calls in total. Under the convention that each call costs at most $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$, $\mathsf{Time}(\mathcal{E}_{\mathsf{ZisK}}) \le 3\kappa_{\max} \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. Each reduction adversary runs $\mathcal{A}$ once and re-runs the extraction chain above its own layer, hence runs in at most $(3\kappa_{\max} + 1) \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. (Strictly, the convention of Definition 2.1 bounds each extractor by the time of the composed adversary that produced its input, which compounds along the recursion to $O(\kappa_{\max}^2) \cdot \mathsf{Time}(\mathcal{A})$; the linear figure uses

[3] This composition is where the first idealization of Remark 2.1 is invoked: a straight-line extractor for a succinct Π_{FINAL} does not exist in the plain model, and composing it with the extractors below requires relativized arguments.

the per-call convention and is part of the idealization of Remark 2.1.) In particular $\mathcal{E}_{\mathsf{ZisK}}$ is PPT whenever $\mathcal{A}$ is, since $2^B \le \mathsf{poly}(\lambda)$. □

2.6. **Discussion.**

Every soundness, binding or extractability term that appears in Theorem 2.21 is either defined in this document, inherited from the backend, or an explicit idealization. Table 3 records the status of each, together with the obligations that a full security argument must discharge.

Term	Status	Where / what remains
$\mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{AIR}}}$	backend assumption	Knowledge soundness of each STARK argument for $\mathcal{R}^{(\tau)}_{\mathrm{AIR}}$ in commit-and-prove form (Definition 1.11); quantified by the backend's own analysis.
$\mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{LEAF}}}$, $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}$, $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{FINAL}}}$	backend assumption	Knowledge soundness of the recursive STARK circuits for Definition 1.12, Definition 1.14, Definition 1.15, including the typing predicate and the vk-dispatch.
straight-line composition	idealization	Remark 2.1 (first item); relativized arguments do not exist [BCG24].
$\mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}$	defined (Definition 2.2)	Standard for Merkle trees over a random oracle.
$\mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}$	idealization	Definition 2.3 and Remark 2.1 (second item). LtHash is only known to be collision resistant, from the short-integer-solution problem, with an inner digest of at least 2λ bits; the rigorous alternative is Remark 2.3.
$\varepsilon_{\mathcal{F}}$	defined (Definition 1.6); proved for LogUp (Lemma 2.9)	Statistical; requires the challenge pair to be a random-oracle output (Definition 2.4). The circuit proves a sub-relation of $\mathcal{R}^{(\tau)}_{\mathrm{AIR}}$ (Section 1.7).
$\mathrm{Balanced}(L)$	proved (Proposition 2.12)	Field-valued; integer balance needs multiplicity range checks (Remark 2.4).

TABLE 3. Definition and obligation ledger for the terms of Theorem 2.21.

The following items are outside the theorem and are recorded as obligations for the sections that build on it.

- **From witnesses to executions.** Theorem 2.21 yields a balanced collection of locally satisfying AIR witnesses for a common public statement. The semantic claim of ZisK, that such a collection encodes one execution of the committed program from the boot state to a halting state whose public outputs are $\boldsymbol{p}$, requires the memory argument, the continuation stitching and the per-chip semantics, each with its own relation and lemma. These compose with Theorem 2.21 by strengthening Local and by reading off the meaning of $\mathrm{Balanced}$ for the specific buses.
- **Integer multiplicities.** See Remark 2.4: a bound on every multiplicity column, in particular on the receiving side of shared lookup tables, is needed to lift field balance to multiset equality.
- **The accumulator.** The linear bound of Proposition 2.12 rests on the preimage awareness of $\mathcal{H}$ (Definition 2.3), the second idealization of Remark 2.1. Discharging it (Remark 2.3) is the main

cryptographic obligation left by this section; the bound that forking gives is valid but concretely loose.

- **Circuit requirements.** The relations of Section 1.6 require the circuits to enforce the typing predicate on children, to hash the type together with the commitment into the challenge contribution, to expose the trace commitment as a public input of the AIR argument, and to have that commitment bind the cached columns, or else to feed the cached-column commitment into the challenge contribution by other means (Section 1.7).
- **Fixed and cached data.** Fixed columns are prescribed by the AIR description, hence by the verification key. Cached columns, for instance the program ROM and precomputed operation tables, are bound through the cached-column commitment carried in the public statement (Definition 1.3): Local_τ only asserts that they open it. Deciding whether the commitment in $\boldsymbol{p}$ is that of the intended program is the task of the consumer of the Final proof, and the equality of the cached columns across all leaves of one type rests on the binding of that commitment, which the theorem does not need and does not charge.
- **Concrete parameters.** With $B = 20$ the running-time factor $3\kappa_{\max}$ is about $3 \cdot 10^6$ and the coefficient of the recursive knowledge-soundness terms is about 10^6; both are polynomial but not small, and any concrete security claim must account for them together with the query bound Q and the caveats of Remark 2.1.
- **Completeness.** The honest pipeline produces, for every balanced collection of at most $2^B - 1$ locally satisfying witnesses, an accepting Final proof; we have not stated this formally, and a complete treatment should.

References

Kad+26. Kadianakis, G., Khovratovich, D., Wagner, B., Zapico, A.: A Vanilla zkVM Architecture, https://github.com/khovratovich/zkvm-ef-security-sprint

GKS23. Grassi, L., Khovratovich, D., Schofnegger, M.: Poseidon2: A Faster Version of the Poseidon Hash Function. In: El Mrabet, N., De Feo, L., and Duquesne, S. (eds.) AFRICACRYPT 23: 14th International Conference on Cryptology in Africa. pp. 177–203. Springer, Cham (2023)

O'C+20. O'Connor, J., Aumasson, J.-P., Neves, S., Wilcox-O'Hearn, Z.: BLAKE3: One Function, Fast Everywhere, (2020)

Nat15. National Institute of Standards and Technology: Secure Hash Standard (SHS). (2015)

Hab22. Haböck, U.: Multivariate lookups based on logarithmic derivatives, https://eprint.iacr.org/2022/1530

Zis25. ZisK: Secure Challenge Derivation in ZisK, https://hackmd.io/SXKiWASTRu2QYgy7B-NGmg

Cla+03. Clarke, D.E., Devadas, S., Dijk, M. van, Gassend, B., Suh, G.E.: Incremental Multiset Hash Functions and Their Application to Memory Integrity Checking. In: Laih, C.-S. (ed.) Advances in Cryptology – ASIACRYPT~2003. pp. 188–207. Springer, Berlin, Heidelberg (2003)

Ajt96. Ajtai, M.: Generating Hard Instances of Lattice Problems (Extended Abstract). In: 28th Annual ACM Symposium on Theory of Computing. pp. 99–108. ACM Press (1996)

BCS16. Ben-Sasson, E., Chiesa, A., Spooner, N.: Interactive Oracle Proofs. In: TCC 2016-B: 14th Theory of Cryptography Conference, Part II. pp. 31–60. Springer, Berlin, Heidelberg (2016)

BCG24. Barbara, A., Chiesa, A., Guan, Z.: Relativized Succinct Arguments in the ROM Do Not Exist, https://eprint.iacr.org/2024/728

DRS09. Dodis, Y., Ristenpart, T., Shrimpton, T.: Salvaging Merkle-Damgård for Practical Applications. In: Advances in Cryptology – EUROCRYPT 2009. pp. 371–388. Springer, Berlin, Heidelberg (2009)

BN06. Bellare, M., Neven, G.: Multi-signatures in the plain public-Key model and a general forking lemma. In: ACM CCS 2006: 13th Conference on Computer and Communications Security. pp. 390–399. ACM Press (2006)

FS87. Fiat, A., Shamir, A.: How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In: Odlyzko, A.M. (ed.) Advances in Cryptology – CRYPTO'86. pp. 186–194. Springer, Berlin, Heidelberg (1987)